

Prosjektrapport

Bacheloroppgave IT og Informasjonssystemer

ProSlides

ProSlides

Utviklet av:

Rikke Hjelseth Larsen - 233645

Martin Mentzen Hansen - 263505

Marcus Woo Hanøy Einarsen - 263549

Sondre Christoffer Mogen - 263532

Fredrik Stalsberg - 263529

1 Forord

Dette prosjektet er gjennomført som en del av bachelorstudiet i IT og informasjonssystemer ved Universitetet i Sørøst-Norge (USN), campus Bø, i emnet BOP3000. Oppgaven ble gitt i samarbeid med RubyNor, noe som har gitt oss en verdifull mulighet til å arbeide med en problemstilling som er relevant for næringslivet. Gjennom studieløpet har vi opparbeidet oss bred kompetanse innen både frontend og backend utvikling, samt databasedesign og systemutvikling. Vi har også fått erfaring med samarbeid i team gjennom ulike gruppeprosjekter, noe som har vært en viktig forutsetning for gjennomføringen av dette prosjektet.

I dette prosjektet har vi utviklet *ProSlides*, en interaktiv web presentasjonsapplikasjon som legger til rette for økt engasjement mellom presentatør og publikum. Løsningen gir publikum mulighet til å delta aktivt under presentasjoner ved å svare på spørsmål og delta i avstemninger (polls) i sanntid. Resultatene blir umiddelbart visualisert, noe som skaper en mer dynamisk og inkluderende presentasjonsopplevelse. Målet med applikasjonen har vært å gjøre presentasjoner mer interaktive, samt gi presentatører bedre innsikt i publikums respons underveis.

Samarbeidet med RubyNor har gitt oss innsikt i hvordan utviklingsprosjekter kan foregå i arbeidslivet, inkludert arbeid med kravspesifikasjoner, tett dialog med oppdragsgiver og bruk av iterative utviklingsprosesser. Dette har bidratt til en bedre forståelse av hva som forventes i en profesjonell arbeidshverdag innen IT-bransjen.

Vi ønsker å takke RubyNor for oppgaven og veiledning underveis, samt veileder Ingrid Sundbø for gode innspill og støtte gjennom prosjektperioden.

Innholdsfortegnelse

1	Forord.....	2
2	Innledning	6
2.1	Bakgrunn	6
2.2	Problemstilling	6
2.3	Bruk av litteratur og internett.....	6
2.4	Sammendrag	7
3	Administrativ beskrivelse	8
3.1	Roller og Ansvarsfordeling	8
3.2	Kommunikasjon og plan	9
3.3	Kanban	10
3.4	USN Bø	11
3.5	Rubynor.....	11
4	Prosjektmetodikk.....	12
4.1	Prosjektledelse.....	12
4.2	Sprints	12
4.3	Scrum.....	13
4.4	Risikoanalyse	14
5	Funksjonelle mål	17
5.1	Funksjonelle krav.....	17
5.2	Interaktive elementer (SRS)	18
5.3	Dynamiske data og variabler	19
5.4	Presentatørvisning.....	19
5.5	Deltakervisning (Second screen)	19
5.6	Andre mål	20
5.7	Sanntidsfunksjonalitet.....	20
6	Arkitektur og design	21
6.1	ER-diagram.....	21
6.2	Nettverksskisse.....	23
6.2.1	Float Chart	24
		25
6.2.2	Use Case Diagram.....	26
6.3	Systembeskrivelse.....	27

6.3.1	React (TypeScript) - frontend	27
6.3.2	Ruby (Rails) - backend	28
6.3.3	UI og design (SHADCN)	29
6.3.4	Canvas-funksjonalitet (Fabric.js)	34
6.3.5	Database (PostgreSQL via Neon)	34
7	Systemarkitektur	37
7.1	Formål	40
7.2	Overordnet arkitektur	40
7.3	Systemkontekst	41
7.4	Frontend-arkitektur	41
7.5	Backend-arkitektur	42
7.6	Databasearkitektur	42
7.7	Sanntidsarkitektur	43
7.8	Nøkkelflyter	43
7.9	Deployering og miljø	44
7.10	Ikke-funksjonelle egenskaper	44
8	Sikkerhet	45
8.1	Autentisering og autorisering	45
8.2	Passordsikkerhet	45
8.3	API og datasikkerhet	45
8.4	Databasesikkerhet	45
8.5	Frontend sikkerhet	46
8.6	Sikring i produksjon	46
9	Testing	47
9.1	Formål med testing	47
9.2	Testoppsettet og gjennomføring	48
9.3	Hva som ble testet	49
9.4	Resultater	49
9.5	Vurdering av testmetoden	49
9.6	Testing Milepæl 3	50
9.7	Brukerhistorier	50
10	Kritisk vurdering og konklusjon	54
10.1	Vurdering	54

10.2	Kritisk vurdering.....	54
11	Bruk av KI.....	56
11.1	KI Verktøy som har blitt brukt:	56
12	Utviklingsverktøy	57
13	Ressurser	59
13.1	Brukere for testing av applikasjonen.....	59
13.1.1	Guide/forklaring til testing av hovedapplikasjon.....	60
14	Litteraturliste.....	62
15	Vedlegg.....	63

2 Innledning

2.1 Bakgrunn

Grunnen til at vi som gruppe valgte denne oppgaven er fordi vi ville ha en utfordring som ga oss verdifulle erfaringer for prosesser i et reelt arbeidsmiljø. Ved å kunne delta og lage dette prosjektet fra bunn av får vi sett hvordan prosesser blir gjort ved et prosjekt gitt av en oppdragsgiver (Rubynor). Vi har hatt tett dialog med oppdragsgiver og hatt statusmøter annenhver fredag.

2.2 Problemstilling

Hvordan kan vi utvikle et webbasert presentasjonsverktøy som både gir brukeren full frihet til å utforme slides visuelt og samtidig legger til rette for aktiv publikumsinvolvering under presentasjonen?

2.3 Bruk av litteratur og internett

I starten av prosjektfasen gikk det tid til forberedelser av arbeidet som skulle gjøres. Under ser vi en tabell med ressurser og måten de ble brukt på for å styrke forberedelsene til utviklingen av prosjektet.

Ressurs	Beskrivelse
YouTube	YouTube ble brukt for å forbedre forståelsen av programmeringsspråket Ruby, med en rekke kurs videoer ble forståelsen bedre, og startfasen av utviklingen lettere.
Google	Google ble brukt betraktelig mye for å finne fram verktøy, ressurser og dokumentasjon som ble nyttig under utviklingen av prosjektet.
Wikis	Wikis instruksjoner ble brukt for oppsett blant annet av authentication, playwright testing, SHADCN, Fabric.js , Neon migrasjon. Kamal deployering og mer.

W3Schools	W3Schools ble brukt gjennom hele prosjektet for guider av Ruby og React.
-----------	--

2.4 Sammendrag

Spleiselaget.no er en digital læringsplattform som benyttes til å gjennomføre interaktive presentasjoner i skolesammenheng. Løsningen gjør det mulig for elever å delta aktivt via mobiltelefon, blant annet ved å svare på spørsmål underveis i presentasjonen (second screen).

Dagens løsning er basert på forhåndsdefinerte maler. Dette gir begrenset fleksibilitet i utforming av innhold og krever at presentasjoner følger faste strukturer.

Vi ønsker nå å utvikle en ny og mer fleksibel presentasjonseditor basert på prinsippet:

WYSIWYG – “What You See Is What You Get”

Brukeren skal kunne bygge presentasjoner visuelt, mer tilsvarende PowerPoint eller Google Slides, samtidig som interaktive funksjoner videreføres og forbedres.

3 Administrativ beskrivelse

3.1 Roller og Ansvarsfordeling

Rollene som Prosjektleder og Scrum Master var de mest fastsatte rollene, og disse hadde et mer ansvar for å styre prosjektets administrative aspekter, for å sikre effektiv bruk av Scrum.

Utenom disse faste rollene, var det et mål at alle skulle få mulighet til å jobbe på tvers av hele teknologistakken (frontend og backend), slik at vi alle satt igjen med verdifull kompetanse. For å sikre fremdrift på kritiske leveranser fordelte vi likevel primære spesialiseringsområder innenfor det tekniske. Tabellen nedenfor gir en god oversikt over både faste administrative roller og de tekniske hovedområdene hvert medlem hadde fokus på.

Navn	Rolle (Administrativt)	Primær Spesialisering (Hovedområder)	Fellesansvar og Nøkkelteknologi
Martin Mentzen Hansen	Prosjektleder, Utvikler	Live funksjonalitet, Databasearkitektur.	Fellesansvar: Utvikling av API-endepunkter (Ruby on Rails) og Frontend-komponenter (React). Primærfokus: Sanntidskommunikasjon (Action Cable), Datamodell (PostgreSQL/Neon). Resultattavle(live)
Marcus Woo Hanøy Einarsen	Scrum Master, Utvikler	Presentasjons editor, frontend arkitektur, scrum	Fellesansvar: Utvikling av API-endepunkter (Ruby on Rails) og Frontend-komponenter (React). Primærfokus: WYSIWYG-editor (Fabric.js), Redigeringsflyt.

Fredrik Stalsberg	Utvikler, UI-Design, Sikkerhet	Autentisering, UI & UX design, Testing (PlayWright E2E)	Fellesansvar: Utvikling av API-endepunkter (Ruby on Rails) og Frontend-komponenter (React). Primærfokus: Brukerautentisering/Sikkerhet, Designsystem (SHADCN), E2E Testrutiner.
Sondre Christoffer Mogen	Utvikler	Backend API og Forretningslogikk	Fellesansvar: Utvikling av API-endepunkter (Ruby on Rails) og Frontend-komponenter (React). Primærfokus: API-struktur, Validering av data og mobilkompatibilitet, Polls og avstemninger
Rikke Hjelseth Larsen	Utvikler	Interaktive elementer	Fellesansvar: Utvikling av API-endepunkter (Ruby on Rails) og Frontend-komponenter (React). Primærfokus: Verktøy og interaksjoner i presentasjons-editor

3.2 Kommunikasjon og plan

Gjennom hele prosjektet har vi benyttet Scrum-metodikken for å sikre strukturert kommunikasjon og god planlegging.

Internt i gruppen holdt vi to faste ukentlige møter for statusoppdateringer, utfordringsdiskusjoner og detaljert oppgaveplanlegging for de kommende sprintene. For koordinering og raske avklaringer tok vi i bruk digitale verktøy som Discord og Kanban-tavlen på GitHub. Eksternt var kommunikasjon gjort gjennom statusmøter annenhver uke med oppdragsgiver Rubynor via Google Meet. Dette sikret at vi kontinuerlig presenterte fremdrift, og mottok verdifulle tilbakemeldinger og diskuterte nødvendige endringer for å møte de funksjonelle kravene.

Tekst	Video	Prosjekttavle
Discord	Discord	Kanban board(GitHub)
Mail	Google Meet	Notion
Snapchat		Google docs
Canvas		
Google forms		
Google docs		

3.3 Kanban

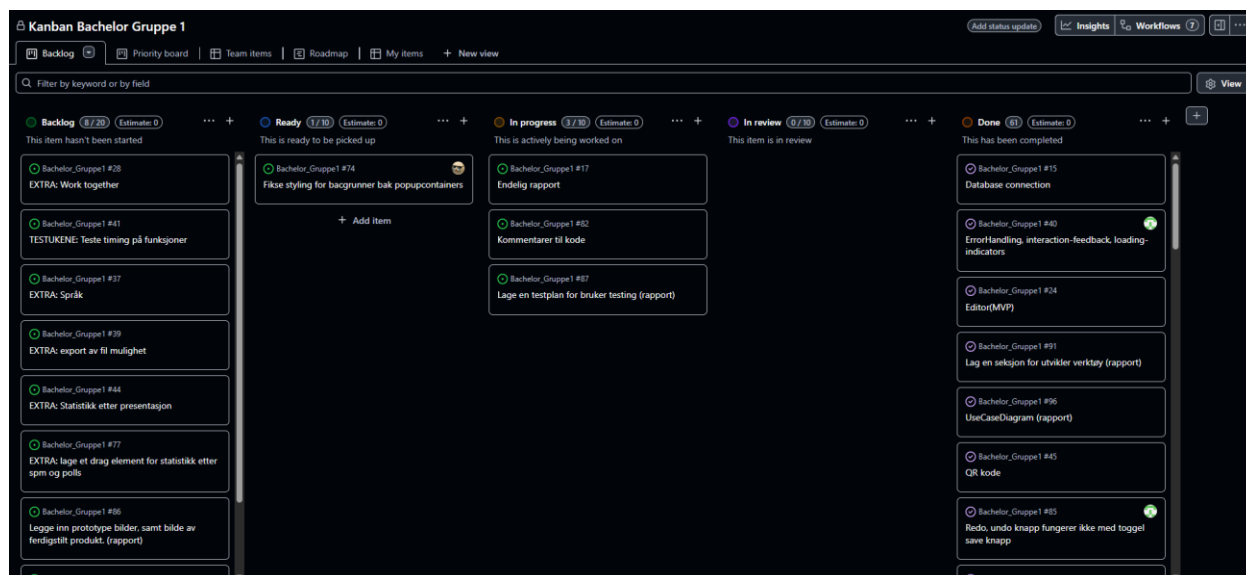
For å sikre god struktur og oversikt over prosjektarbeidet, tok vi i bruk et Kanban board i GitHub gjennom hele prosjektperioden. Dette visuelle verktøyet var essensielt for å implementere vårt agile rammeverk (SCRUM) på en effektiv måte.

Kanban tavlen var delt inn i dedikerte kolonner som ga oversikt over oppgavens progresjon, fra start til slutt.

- **Backlog:** Samlet oversikt over alle identifiserte oppgaver og funksjonalitet.
- **Ready to be picked up:** Oppgaver som var ferdigdefinert og klare for implementasjon.
- **In progress:** Oppgaver som et gruppemedlem aktivt jobbet med.
- **In review:** Ferdig implementerte arbeid som ventet på gjennomgang og testing.
- **Done:** Fullførte og godkjente oppgaver.
-

Bruken av Kanban skapte en god oversikt over både gjenstående oppgaver, pågående arbeid og det som var klart for kvalitetssikring. Denne klare visualiseringen av arbeidsprosessen gjorde oppgavefordelingen svært effektiv. Hvert gruppemedlem kunne enkelt se hvilke oppgaver som var tilgjengelig i køen, noe som la til rette for et dynamisk samarbeid og sikret jevn fremdrift mot prosjektmålene. Under ser vi hvordan Kanban tavlen ser ut:

(Kanban)



3.4 USN Bø

Ingrid Sundbø har vært veileder gjennom bachelorprosjektet vårt. Gjennom prosjektets varighet har vi fått tilstrekkelig med veiledning og oppfølging.

Navn	Stilling	Sted	Kontakt info
Ingrid Sundbø	Veileder	USN (Bø)	ingrid.sundbo@usn.no

3.5 Rubynor

Rubynor har vært vår oppdragsgiver gjennom prosjektets varighet. Vi har hatt tettest dialog med Pål André Sundt, CTO for Rubynor. Vi har hatt statusmøter annenhver uke.

Navn	Stilling	Sted	Kontakt info
Pål André Sundt	CTO	Rubynor (Skien)	pal@rubynor.com

4 Prosjektmetodikk

4.1 Prosjektledelse

Vi har valgt å dele inn oppgaver av ulike sider av prosjektet, dette er for å gi alle en mulighet for å jobbe med hvert aspekt av denne oppgaven. Vi ville at alle kunne være innom det meste så alle kan ta med seg gode erfaringer mot en fremtidig jobb.

Arbeidsprosessen ble organisert med Scrum og sprintbasert utvikling, hvor vi jobbet iterativt med planlegging, implementasjon og testing. Dette bidro til tett samarbeid i gruppen og gjorde det enklere å tilpasse løsningen underveis.

Siden starten av BOP-perioden har vi internt hatt to ukentlige møter, samtidig har vi hatt møter med oppdragsgiver (Rubynor) annenhver uke. I disse møtene har vi presentert framgang i prosjektet, diskutert mulige endringer og fått tilbakemeldinger for å sikre at løsningen vår oppfyller kravene til funksjonalitet og brukeropplevelse.

4.2 Sprints

Prosjektet er delt opp i fire forskjellige sprinter. Sprint periodene er tilpasset tidsrammen vi har fått. Under ser du hvilken sprint, og hva den inneholder.

Sprint 1 (Foranalyse) - Sette i gang med prosjekt. Sette opp nødvendig IDE, rammeverk og midler for å kunne starte prosjekt. Veldig viktig at vi setter oss inn i det nye kodespråket(Ruby). Database(PG) må bli satt opp med riktig connection.

Sprint 2 (UI, UX, datastruktur) - UI må få en fast struktur. UX må være interaktivt og intuitivt. Må også passe på universell utforming.

Sprint 3 (Editor, interaktivitet) - Editoren kommer til å være helhelten i prosjektet, derfor får den en hel sprint på seg. Her er det veldig viktig å få på plass all funksjonalitet. Det er i selve editoren som alt kommer til å skje. Det vil også være tilhørende verktøy og funksjoner. Her

gjelder det å lage en strukturert side. Vi må passe på at det ikke blir for mye små ting på siden, og heller ikke for lite. Den skal både være oversiktlig, men også med tilhørende funksjoner (verktøylinje, polls, ordsky, slides osv.).

Sprint 4 (Testing) - Testing av prosjekt vil bli gjort av brukere som tester nettsiden, brukerhistorier og noe selvtesting av prosjektteamet. Eventuelle feil eller mangler vil bli fikset fortløpende.

Rapport - Gjøres gjennom hele prosjektets levetid.

Sprint	Hva	Uke
Start(S1)	Foranalyse	5-7
Design(S2)	UI, UX, datastrukturer	8-10
Implementasjon(S3)	Editor, interaktivitet og sanntidsfunksjonalitet	11-17
Testing(S4)	Feilretting og stabilisering	18-20
Rapport	Skriving av rapport	5 - levering av produkt
Levering av endelig produkt	Optimalisering, feilretting, endringer	21 (19.05. Kl.1400)

4.3 Scrum

Vi bruker SCRUM som vårt agile rammeverk for å kunne løse oppgaver på en mer strukturert måte. Det blir også brukt for planlegging og møtevirkosomhet. Vi har laget et lite møtereferat til alle møtene vi har hatt. Det gjør det enklere for oss å følge med på progresjonen vår og om alle er på samme side. Eventuelle feil, oppgavefordeling, mangler eller nye ting er det som oftest blir diskutert.

4.4 Risikoanalyse

Risikoanalyse er en metode som brukes til å forstå og beskrive potensielle utfordringer og usikkerhetsfaktorer knyttet til en bestemt aktivitet, i dette tilfellet et prosjekt. Den inneholder vurderinger av hvor sannsynlig en hendelse er, eventuell påfølgende konsekvens, og brukes som grunnlag for å ta beslutninger om risiko, og planlegge tiltak for å redusere dem. (Aven, 2026)

I dette prosjektet har vi utarbeidet risikoanalysen i startfasen for å få en bedre oversikt over sannsynlige utfordringer knyttet til utviklingen av ProSlide. Siden prosjektet vårt innebærer det å utvikle en interaktiv webapplikasjon med sanntidsfunksjon, stiller dette krav til både teknisk kompetanse, samarbeid i prosjektgruppen og god planlegging. Risikoanalysen vår består derfor av både tekniske risikoer, som bruk av ny teknologi, og organisatoriske risikoer som kommunikasjon, rollefordeling og tidsstyring.

Vi har valgt å gjennomføre risikoanalysen ved å bruke en enkel, kvalitativ metode, hvor risikoene ble vurdert etter “lav”, “moderat” eller “høy” ut fra sannsynlighet og konsekvens. Denne metoden ga en enkel oversikt, og var en praktisk måte å identifisere og prioritere risikoer gjennom hele prosjektet.

Hendelse	Beskrivelse	Sannsynlighet	Konsekvens	Tiltak
Tidsmangel	Mangel på tid til å fullføre oppgaver innen tiden vi har satt	Moderat	Høy	Utarbeide en tydelig tidsplan, sørge for at det viktigste er på plass før vi begynner med pirkearbeid.
Uklare roller og ansvar	Gruppemedlemmer er usikre på hvem som skal gjøre hva, noe som kan skape forvirring	Moderat	Høy	Fordele tydelige roller og ansvar tidlig i prosjektet
Manglende erfaring med Ruby On Rails	Ruby On Rails er noe ingen i gruppen har arbeidet noe særlig med før.	Høy	Høy	Sette oss inn i språket tidlig, og klare å diskutere innen gruppen da det kan være lettere å forstå og noen i gruppen kan ha forstått noe

				noen andre ikke har.
Tekniske problemer	Manglende tilgang til nødvendige verktøy eller teknisk utstyr	Lav	Moderat	Sørge for alternative løsninger og oppdatere programvare på forhånd.
Teknologisk kompleksitet	Utvikling av avansert webbasert applikasjon som kombinerer flere teknologisk krevende komponenter	Moderat	Høy	Avgrense funksjonalitet
Uklare krav	Uklare krav kan føre til forvirring og eventuelt ekstraarbeid	Moderat	Høy	Vi skal være flinke til å spørre både de andre gruppemedlemmene og vår kontaktperson hos Rubynor om det er noe vi er usikre på.
Mangel på kommunikasjon	Mangel på kommunikasjon mellom gruppemedlemmer, som kan føre til misforståelser	Moderat	Høy	Regelmessige gruppemøter, bruke digitale verktøy som Discord, Snapchat og lignende.
Lav motivasjon hos medlemmer	En eller flere i gruppen mister engasjement under prosjektet	Moderat	Moderat	Lage klare mål og feire små milepæler for å opprettholde engasjement.
Uventede eksterne faktorer	Sykdom, press eller andre forstyrrelser som påvirker gruppens framdrift	Lav	Høy	Ha en fleksibel, men tydelig tidsplan, og ha backup-planer for kritiske oppgaver.

Risikoen knyttet til bruk av et nytt applikasjonsrammeverk viste seg å være relevant i prosjektets startfase. Ingen i gruppen hadde tidligere erfaring med Ruby on Rails, noe som gjorde at vi måtte sette av tid til å tilegne oss nødvendig kunnskap om rammeverket. Dette ble håndtert ved å bruke sprint 1 (foranalyse) til opplæring og utforsking av teknologien. Et viktig mål var ikke bare å

lære det grunnleggende, men også å sikre at alle i teamet følte seg trygge på hvordan Ruby on Rails fungerer. Selv om innlæringsfasen krevde tid, viste rammeverket seg å ha et brukervennlig utviklingsmiljø og effektive løsninger som gjorde videre koding enklere.

Risikoen knyttet til samarbeid og kommunikasjon i gruppen viste seg å være veldig relevant. Vi har organisert prosjektet slik at alle deltakere jobbet på flere deler av systemet, noe som ga oss god læring, men også førte til utfordringer med koordinering. Resultatet av dette var for eksempel merge conflicts, og situasjoner hvor arbeidet måtte gjøres på nytt. Vi har også identifisert samarbeid som en potensiell utfordring, men tiltakene våre kunne vært mer konkrete, blant annet gjennom tydeligere ansvarsfordeling og strengere rutiner for bruk av versjonskontroll.

Risikoen for tidsmangel ble også tidels realisert, spesielt i implementasjonsfasen. Systemets omfang, særlig knyttet til sanntidsfunksjon, gjorde at enkelte oppgaver tok mer tid enn forventet. Dette førte til at framdriftsplanen måtte justeres, da ved å utvide implementasjonsfasen. Samtidig fungerte det å prioritere kjernefunksjonalitet som et effektivt tiltak for å sikre at de viktigste delene av systemet ble ferdigstilt.

Når det gjelder tekniske risikoer knyttet til systemets stabilitet og sikkerhet, opplevde vi færre problemer enn forventet. Det som bidro til å redusere sannsynligheten for alvorlige feil var valg av etablerte teknologier og rammeverk.

I ettertid har vi sett at noen risikoer ikke ble godt nok identifisert i den opprinnelige analysen. Dette gjelder spesielt utfordringer knyttet til versjonskontroll og koordinering av kodearbeid i gruppen.

Samlet sett har risikoanalysen gjort oss mer bevisste rundt potensielle utfordringer og gjort det enklere å håndtere problemer som oppstod. Erfaringene fra dette prosjektet viser viktigheten av god planlegging, realistiske vurderinger og tydelig struktur i samarbeid og arbeidsprosesser.

5 Funksjonelle mål

Alt som er nevnt i punkt 3, er tatt direkte fra oppgavebeskrivelsen som vi har fått av Rubynor. Se vedlegg A (oppgavebeskrivelse).

5.1 Funksjonelle krav

WYSIWYG Slide-editor

Brukeren skal kunne:

- Opprette, redigere og slette slides
- Endre rekkefølge på slides (drag & drop)
- Se live forhåndsvisning av presentasjonen
- Redigere innhold direkte i sliden (inline editing)

Editoren skal være elementbasert, ikke malbasert.

Brukeren skal kunne dra inn elementer fra en verktøymeny, for eksempel:

- Tekstboks
- Tittel
- Punktliste
- Bilde
- Bakgrunn
- Video
- Interaktivt spørsmål
- Avstemning (multiple choice)
- Fritekstspørsmål
- Ordsky
- Resultatvisning (graf/tabell)

Hvert element skal kunne:

- Flyttes fritt (drag & drop)
- Endre størrelse
- Redigeres direkte
- Konfigureres via egenskapspanel

5.2 Interaktive elementer (SRS)

Plattformen skal støtte second screen-funksjonalitet:

- Deltakere får tilgang via mobiltelefon (QR-kode, kode)
- Deltakere kan:
 - Svare på multiple choice
 - Sende inn fritekst
 - Delta i polls
- Svar lagres og knyttes til aktiv presentasjon

Resultater skal kunne:

- Viser i sanntid
- Presenteres som:
 - Stolpediagram
 - Ordsky
- Brukes dynamisk i senere slides

Eksempel:

En slide kan hente resultater fra en tidligere avstemning (poll) og vise dem automatisk.

5.3 Dynamiske data og variabler

Systemet skal støtte:

- Referanser til dynamiske verdier (f.eks. antall deltakere)
- Mulighet for å bruke svar fra tidligere spørsmål i senere slides
- Et enkelt “variabelsystem” for endring av verdier i eget format.

5.4 Presentatørvsning

Det skal utvikles en webbasert presentatørvsning (gjerne som PWA):

- Navigasjon mellom slides
- Oversikt over antall deltakere
- Start/stopp av spørsmål og polls
- Kontroll over når resultater vises

Prosjektorvsning:

- Mulighet for presentatør å “poppe ut” nytt vindu for fullskjerm visning av presentasjon (til prosjektorvsning).

5.5 Deltakervsning (Second screen)

Nettleserbasert side for visning av presentasjon:

- Viser aktiv slide hos deltaker på PC
- Se aktive interaksjoner som spørsmål og polls
- Se eget resultat/svar først, så resultatsider når presentatør går videre
- Fullskjerm-modus for PC-deltakere
- Mulighet for å forlate økt

En mobiltilpasset side hvor elevene kan:

- Enkelt komme inn via QR-kode
- Se en enkel placeholder-side når ingen interaksjoner er aktivert
- Se aktivt spørsmål
- Spørsmål aktiveres fra serveren med websocket
- Sende inn svar
- Få bekreftelse på innsending

5.6 Andre mål

Brukervennlighet

- Intuitivt og moderne grensesnitt
- Minimal opplæring nødvendig
- Tydelig skille mellom redigering og presentasjon

5.7 Sanntidsfunksjonalitet

- Løsningen må håndtere sanntidsoppdatering (WebSockets e.l.)
- Resultater skal kunne oppdateres uten sideoppdatering

6 Arkitektur og design

6.1 ER-diagram

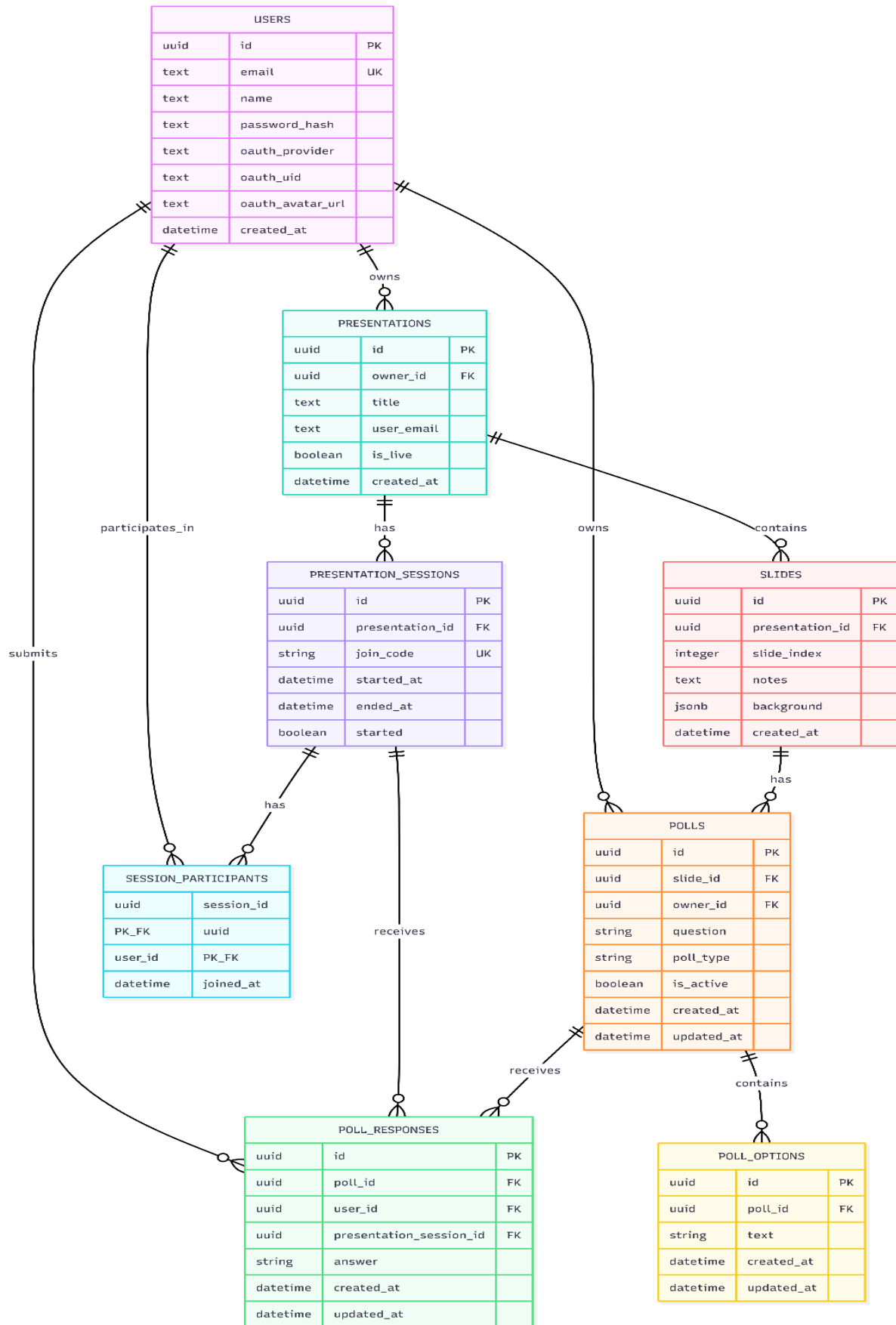
ER-diagrammet viser den overordnede datamodellen i systemet og hvordan sentrale entiteter er relatert til hverandre. Modellen er utformet for å støtte håndtering av brukere, presentasjoner, live-sessions og interaktive elementer som polling og svar.

USERS er en kjerneentitet som representerer systemets brukere. En bruker kan eie flere PRESENTATIONS og POLLS, delta i live-sessions gjennom SESSION_PARTICIPANTS, samt sende inn svar via POLL_RESPONSES.

En PRESENTATION består av flere SLIDES og kan ha flere tilhørende PRESENTATION_SESSIONS, som representerer gjennomføringer av presentasjonen i sanntid. Hver sesjon har flere deltakere og mottar svar fra brukere underveis.

SLIDES fungerer som individuelle sider i en presentasjon og kan inneholde flere POLLS. Hver poll kan flere POLL_OPTIONS og mottar svar gjennom POLL_RESPONSES, som knytter sammen bruker, poll og aktiv session.

Relasjonene i modellen er implementert ved hjelp av primærnøkler og fremmednøkler, som sikrer konsistens og referanseintegritet i databasen. Samlet gir ER-diagrammet en strukturert representasjon av hvordan data organiseres og samhandler i systemet.



6.2 Nettverksskisse

Figuren under viser en overordnet struktur av hvordan brukeren beveger seg gjennom applikasjonen, fra innlogging til ulike funksjoner i systemet.

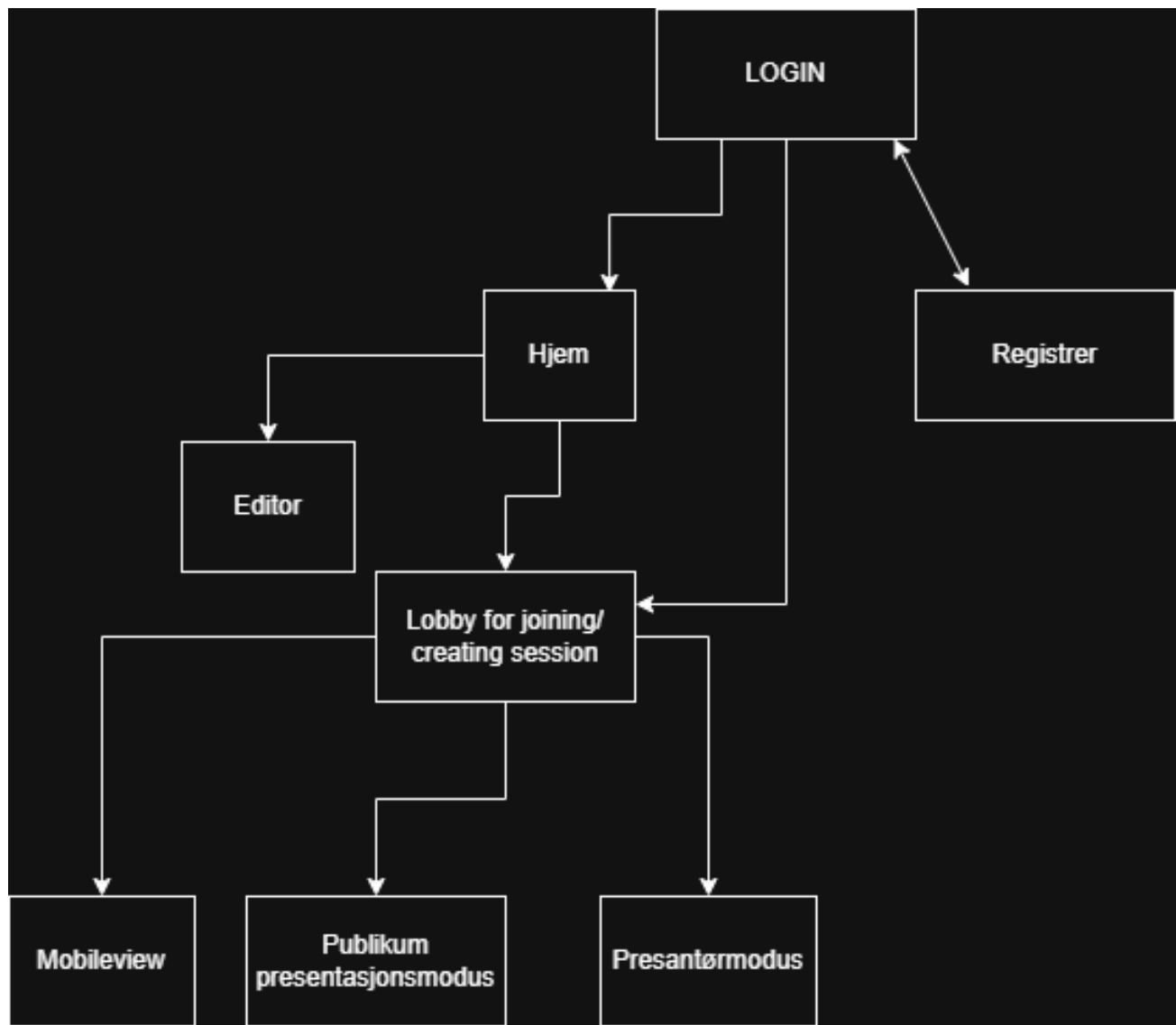
Brukerreisen starter ved login-siden, hvor brukeren enten kan logge inn, navigere til registreringssiden, eller bruke en kode for live session og komme direkte til lobby for live session som gjest bruker.

Fra hjem-siden kan brukeren gå til editoren, hvor presentasjoner opprettes og redigeres. I tillegg har brukeren tilgang til å slutte seg til lobby for live session, som er knutepunktet for systemet. Her kan man enten opprette en ny presentasjons-økt eller bli med i en eksisterende kode.

Lobbyen fungerer som en inngangsport til tre ulike moduser:

- Presentatørmodus, hvor brukeren styrer presentasjonen
- Publikum presentasjonsmodus, hvor deltakeren kan følge presentasjonen og svare på spørsmål og polls.
- mobilvisning , som gir en tilpasset opplevelse for brukere på mobil (second screen)

Denne strukturen legger til rette for en tydelig rollefordeling mellom presentatør og publikum, samtidig som det støttes interaktivitet i sanntid. Flyten i systemet er designet for å være intuitivt, slik at brukeren enkelt kan navigere mellom oppretting, deltakelse og gjennomføring av presentasjoner.

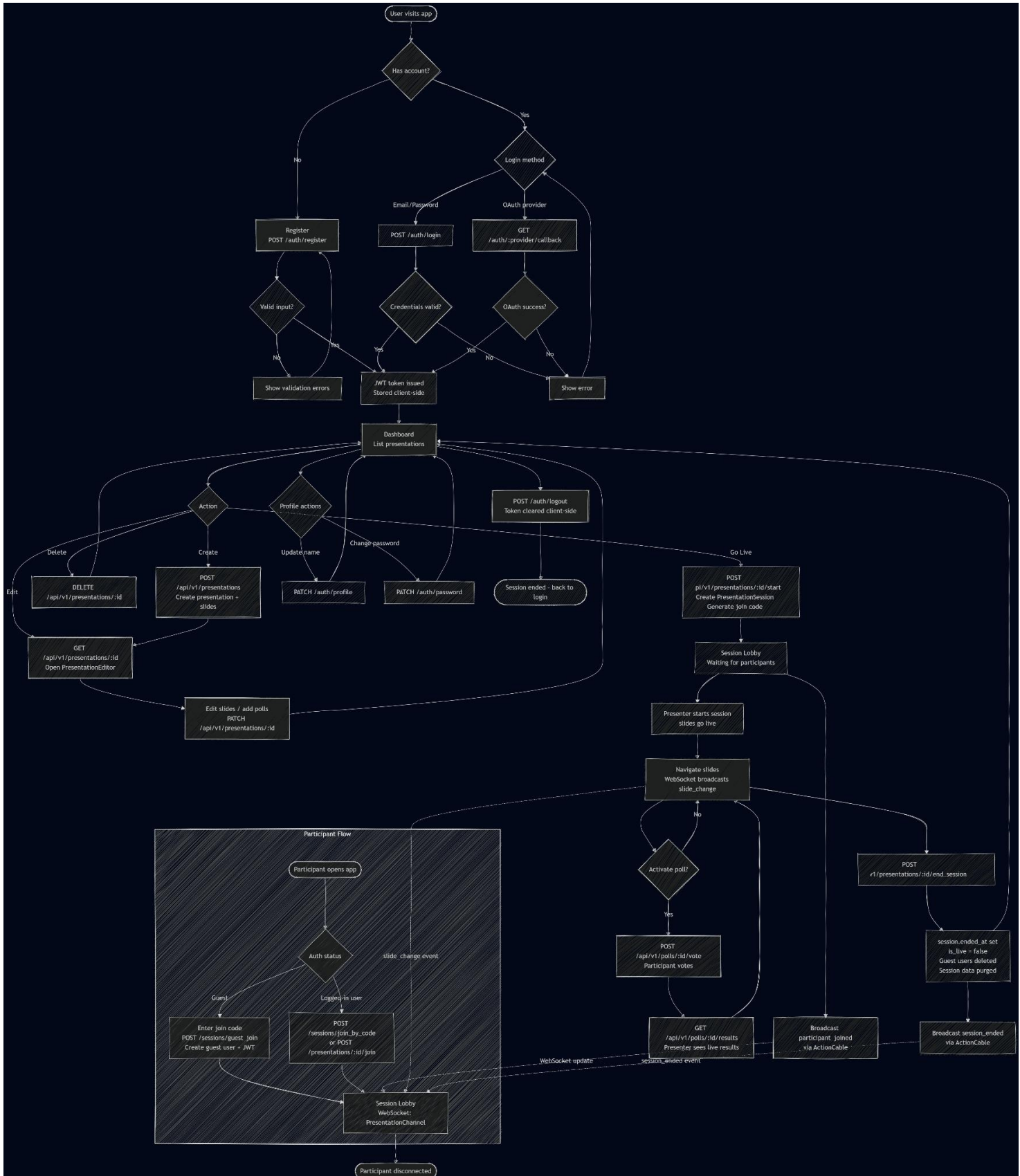


6.2.1 Float Chart

For å visualisere hvordan webapplikasjonen fungerer i sin helhet, har vi laget et flyt-diagram.

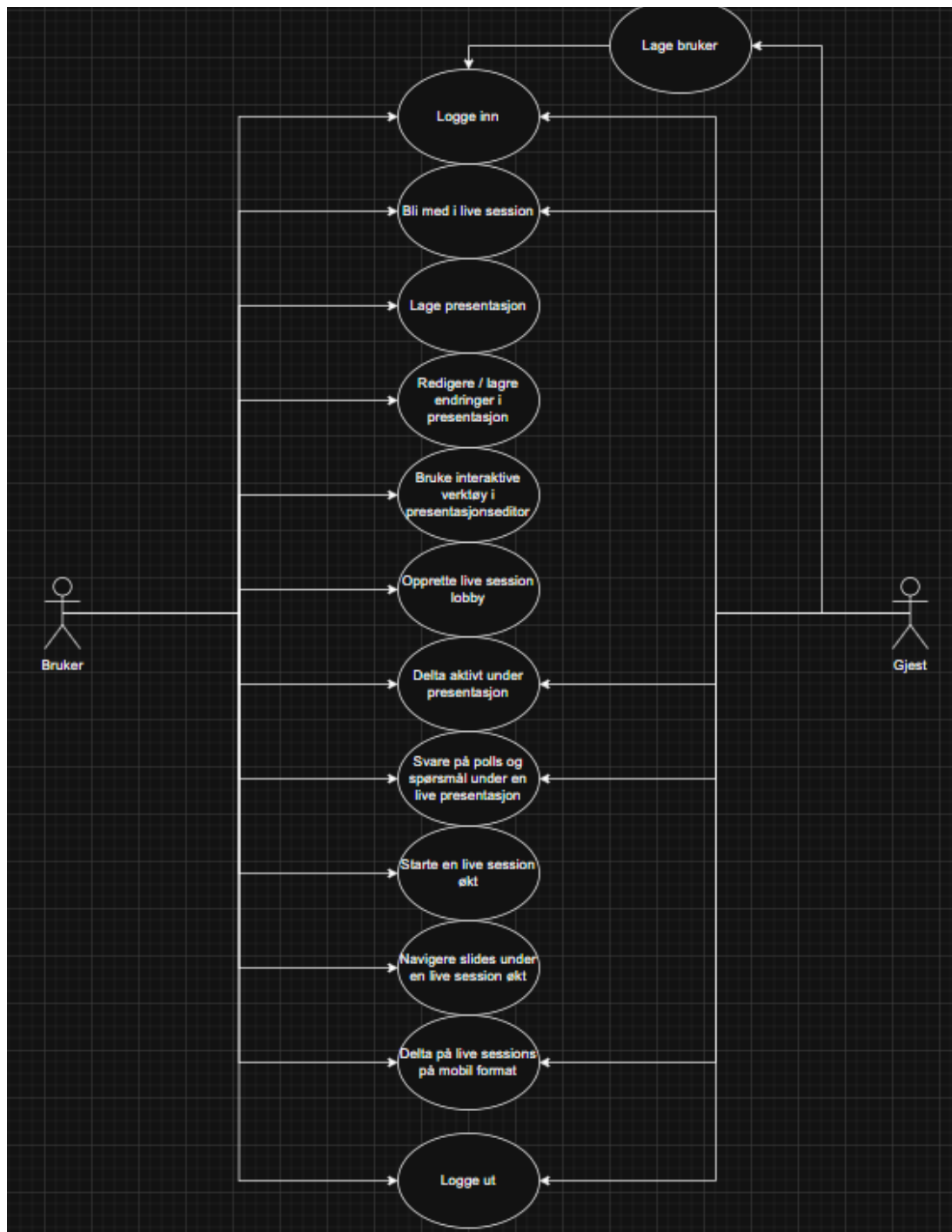
Flyt diagrammet viser prosessen til applikasjonen fra når en bruker/gjest besøker siden og helt til slutt når bruker/gjest avslutter eller logger ut av webapplikasjonen. Se vedlegg B (Float Chart).

Flyt diagrammet ser man under:



6.2.2 Use Case Diagram

Dette er et overordnet diagram for hvordan applikasjonen fungerer for allerede eksisterende brukere (Bruker), og gjester som bare skal delta i en presentasjon (Gjest).



6.3 Systembeskrivelse

I bachelorprosjektet vårt har vi valgt å bygge webapplikasjonen med en moderne teknologi-stack som er brukt mye i dag. Løsningen er delt opp i frontend, backend og database, noe som gjør det enklere å holde oversikt, videreutvikle systemet og fordele ansvar mellom de ulike delene.

Selv om Elm ble anbefalt til oss av oppdragsgiver valgte vi å gå for React frontend. Dette er siden gruppen har tidligere erfaring med React gjennom studiet, noe som hjalp oss gjennom hele prosjektperioden. React gir oss mulighet til å lage et raskt og responsivt brukergrensesnitt. Backend er utviklet i Ruby, og tar seg av logikk, håndtering av forespørsler og kommunikasjon med databasen. Til lagring av data bruker vi PostgreSQL via Neon, som gir oss en skybasert fleksibel database.

I tillegg har vi brukt noen verktøy for å forbedre brukeropplevelsen. SHADCN er brukt for å lage et ryddig og konsistent design, mens [Fabric.js](#) er brukt til canvas-funksjonalitet i presentasjonsdelen av applikasjonen. Dette gjør det mulig å jobbe med interaktive elementer direkte i nettleseren.

Alt i alt gir dette oss en løsning som er strukturert, moderne og enkel å jobbe videre med. Videre forklarer vi hver del av systemet i mer detalj.

6.3.1 React (TypeScript) - frontend

Grunnen til at TypeScript ble valgt over JavaScript er for at det blir bedre kontroll og færre feil i større prosjekter som dette. TypeScript gjør at vi må definere typer, som kommer på både godt og vondt, ulempen med det er at vi må definere typer som tar lengre tid i seg selv og er en litt større læringskurve, men dette er en investering og gjør at det blir lettere å lese/forstå koden man ikke har skrevet selv. Alt i alt er TypeScript tryggere, mer strukturert og skalerbart.

Frontend-delen av applikasjonen er utviklet med React, og er spesielt tilpasset presentasjonsverktøyet vi har laget i bachelorprosjektet. Målet har vært å lage et brukervennlig og interaktivt grensesnitt både for presentatører og publikum.

Ved å bruke React sin komponentbaserte struktur har vi delt opp løsningen i mindre, gjenbrukbare deler, som for eksempel:

- Slides / presentasjonsvisning
- Kontrollpanel for presentatør (polls og spørsmål)
- Interaktive elementer for publikum

Dette gjør det enklere å utvikle og videreutvikle funksjonalitet, samtidig som koden holdes ryddig og oversiktlig.

En viktig del av frontend er håndtering av state, som bruker til å holde styr på:

- Hvilken slide som vises
- Innholdet i presentasjonen
- Interaksjoner fra publikum
- Status på live sessions

Når state oppdateres, sørger React automatisk for at brukergrensesnittet oppdateres i sanntid, noe som er avgjørende for en god presentasjonsopplevelse.

Vi har også lagt vekt på:

- Rask respons og flyt i presentasjonen (ingen behov for refreshing av siden)
- Tydelig skille mellom presentatør og publikumsvisning
- Integrasjon mot backend for live oppdateringer.

6.3.2 Ruby (Rails) - backend

Backend er utviklet med Ruby, ved bruk av rammeverket Ruby on Rails. Rails gjør det raskere å utvikle robuste webapplikasjoner ved å tilby ferdige løsninger for blant annet routing, databasehåndtering og strukturering av kode. Dette lar oss fokusere mer på funksjonalitet enn på grunnleggende oppsett.

Backend fungerer som bindeleddet mellom frontend og databasen, og håndterer all forretningslogikk i systemet. Den tar imot forespørsler fra frontend, prosesserer dem og returnerer relevante svar.

Her er noen sentraler oppgaver backend håndterer:

- API-enderpunkter som frontend kommuniserer med
- Validering og lagring av data i databasen
- Håndtering av brukere og eventuelle rettigheter
- Strukturering av applikasjonslogikk på en oversiktlig måte

6.3.2.1 Realtime-funksjonalitet (live sessions)

En viktig del av applikasjonen vår er støtte for live sessions mellom presentatør og publikum.

For å få dette til har vi brukt realtime-funksjonalitet i Rails, gjennom WebSockets (Action Cable).

Dette gjør det mulig å oppdatere innhold i sanntid uten at brukeren må laste siden på nytt.

- Presentatøren og publikum får spørsmål og polls oppdatert live.
- Alle deltakere i en session ser samme innhold samtidig.

Fordeler med denne løsningen er:

- Gir en mer interaktiv og engasjerende opplevelse
- Reduserer forsinkelser sammenlignet med tradisjonelle API-kall
- Gjør applikasjonen mer egnet for presentasjoner og samarbeid i sanntid

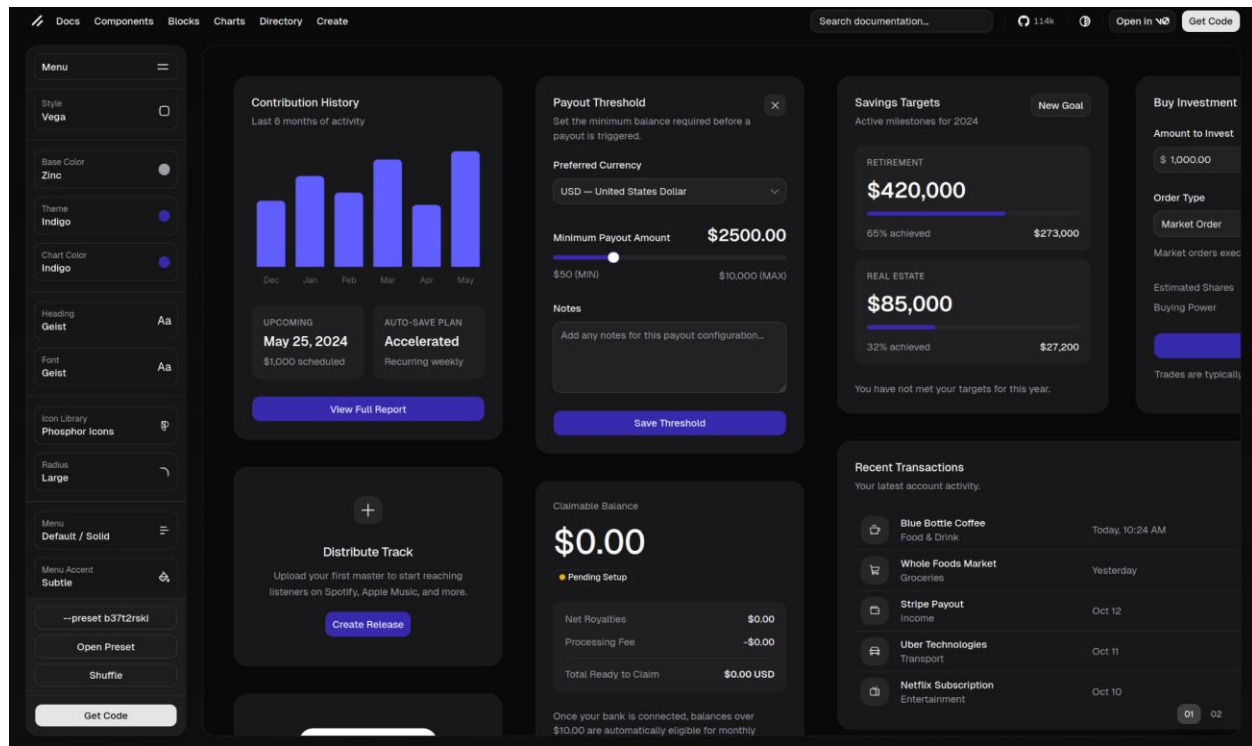
6.3.3 UI og design (SHADCN)

For design og brukergrensesnitt har vi brukt SHADCN, som er et komponentbibliotek som gjør det enklere å lage et konsistent og moderne design.

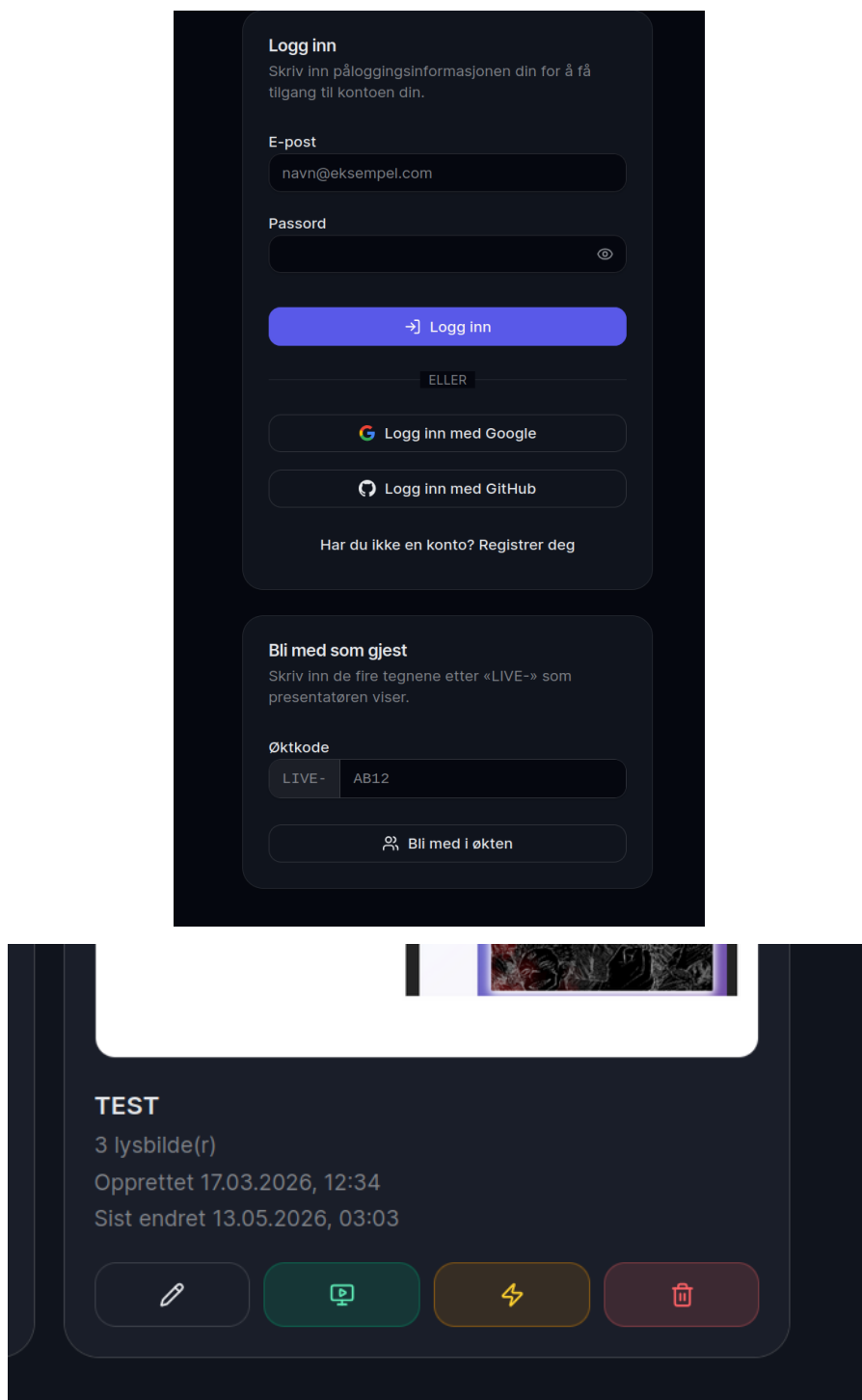
Dette hjelper oss med:

- Ferdigstilte UI-komponenter (knapper, cards, modaler osv.)
- Konsistent stil gjennom hele applikasjonen
- Mindre tid brukt på å lage design fra bunnen av
- ShadCN preset-kode: --preset aJNqUbY:
 - Stil: Vega
 - Farger: Indigo, for å passe sidens exallidraw design inspirasjon
 - Border-radius: Large (Rundere)

(ShadCN preset i deres verktøy)



(Implementasjon av --preset aJNqUbY i prosjektet)

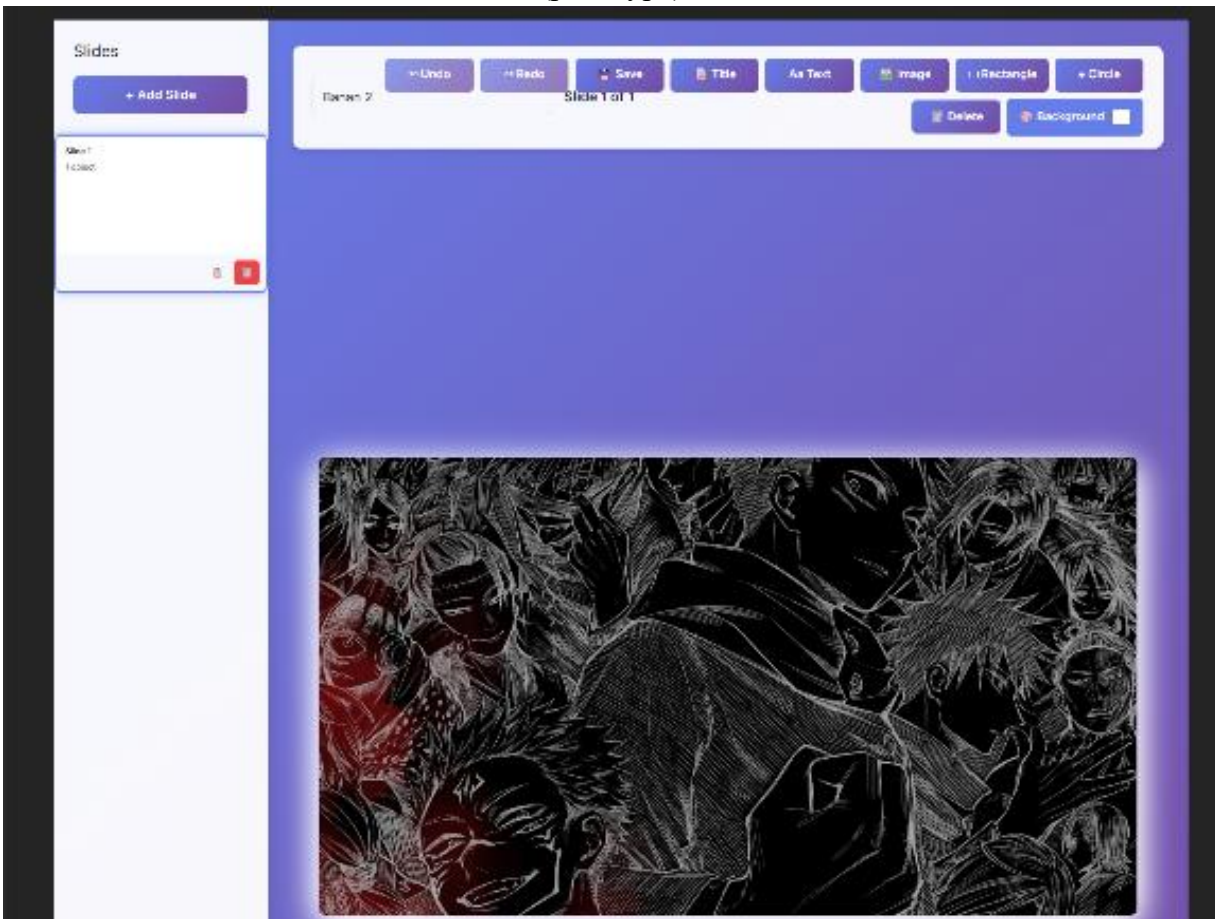


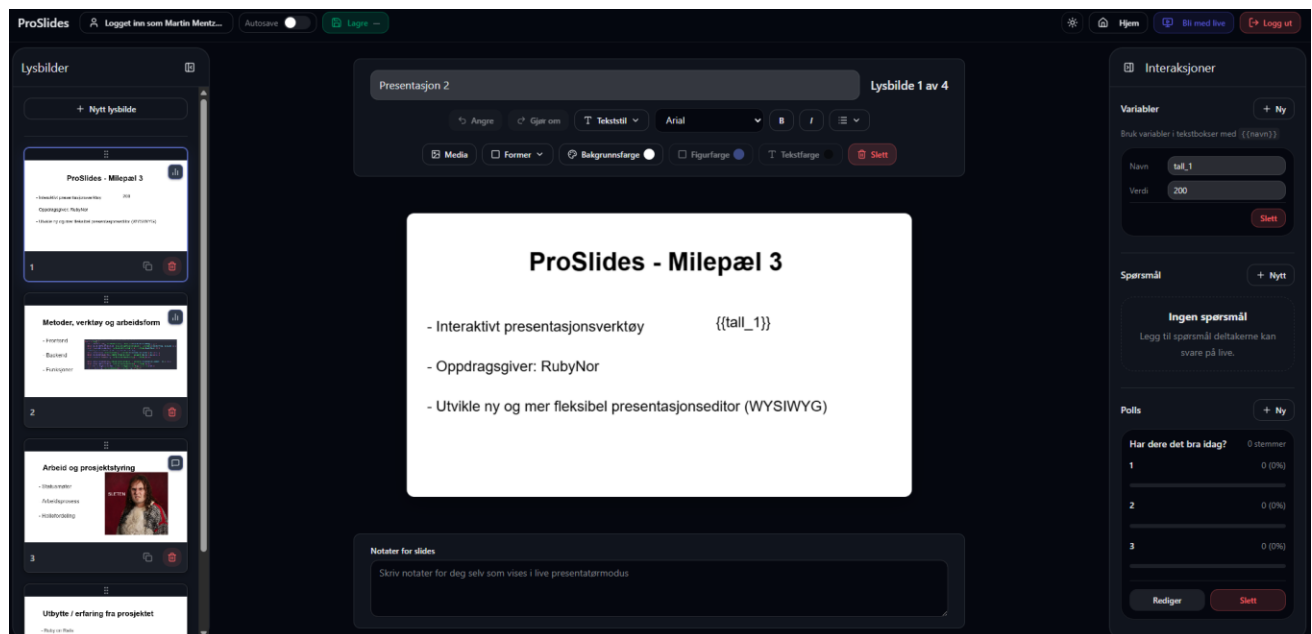
Ved å bruke SHADCN får applikasjonen et profesjonelt utseende uten at vi må implementere alt selv.

6.3.3.1 Prototype vs. ferdig produkt

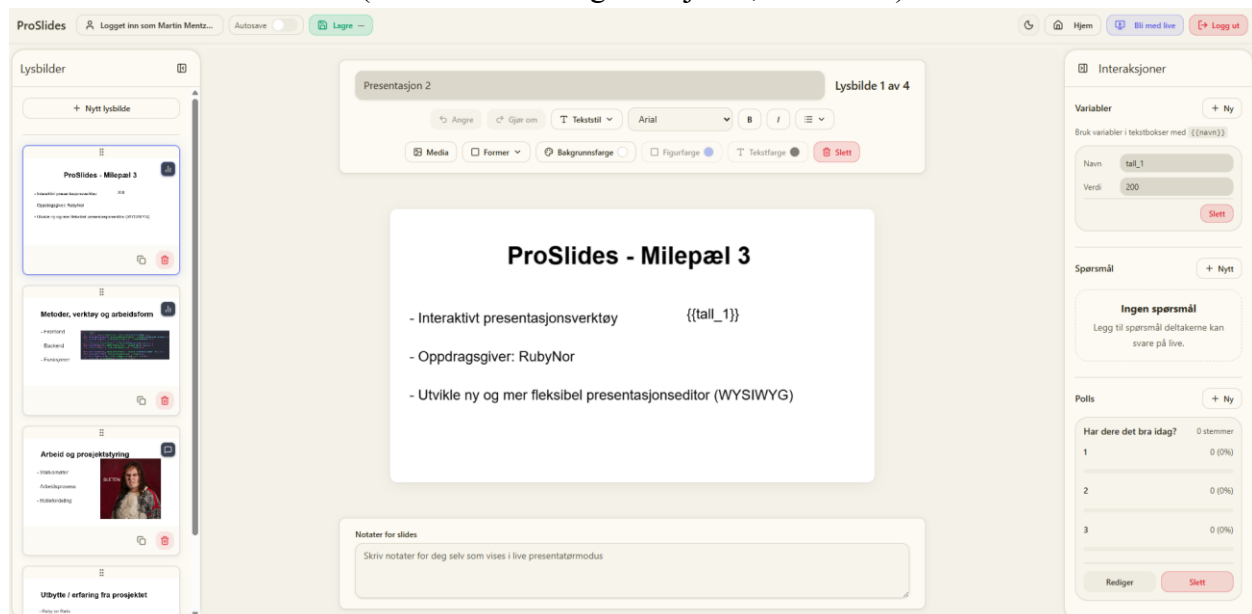
Under ser vi vår første implementasjon av presentasjons- editoren (prototypen) ved bruk av [fabric.js](https://github.com/dnclabs/fabric.js), og etter at vi implementerte SHADCN og alle funksjoner som skal være i editoren. Her ser vi et klart skille og hva som er mulig med SHADCN.

(prototype)





(etter SHADCN og funksjoner, darkmode)



(med lightmode)

6.3.4 Canvas-funksjonalitet ([Fabric.js](#))

I presentasjonsdelen av applikasjonen har vi brukt [Fabric.js](#) for å implementere canvas-funksjonalitet. Dette gjør det mulig å jobbe med grafiske elementer direkte i nettleseren.

Med [Fabric.js](#) kan brukeren:

- Tegne og manipulere objekter
- Flytte, rotere og endre størrelse på elementer
- Lage mer interaktive presentasjoner

Dette gir en mer avansert og engasjerende brukeropplevelse, spesielt sammenlignet med statisk innhold.

6.3.5 Database (PostgreSQL via Neon)

Databasen i systemet er bygget med PostgreSQL og hostet via Neon. Vi har valgt en relasjonsdatabase fordi strukturen i applikasjonen vår består av mange tydelige relasjoner mellom ulike typer data, som brukere, presentasjoner, slides og interaksjoner.

ER-diagrammet vårt gir en oversikt over hvordan disse dataene henger sammen.

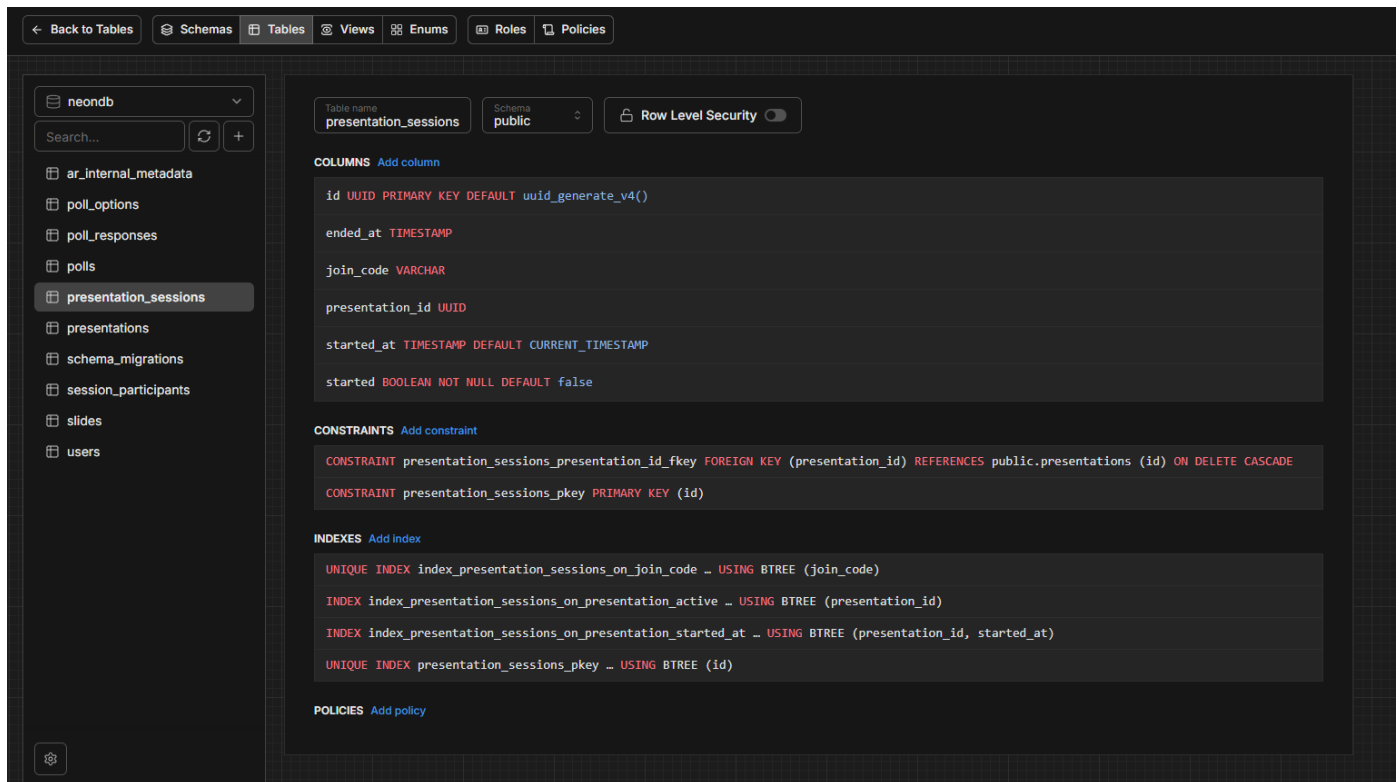
På et overordnet nivå er systemet bygget opp rundt noen sentrale hovedtabeller:

- Users: inneholder informasjon om brukere, inkludert innlogging (både vanlig og OAuth)
- Presentations: representerer presentasjoner oppretter av brukere
- Slides: hver presentasjon består av flere slides
- Slide_elements: elementene som vises på hver slide (tekst, bilder, osv.) lagret fleksibelt med JSON

Dette gjenspeiler strukturen i presentasjonsverktøyet, hvor en bruker lager en presentasjon som igjen består

av flere slides med ulike visuelle elementer

(Neon).



6.3.5.1 Kobling til interaktivitet og live sessions

Databasen støtter også den interaktive delen av applikasjonen:

- Presentation_sessions brukes for å håndtere live presentasjoner
- Session_participants kobler brukere til en aktiv session
- Polls, poll_options og poll_responses gjør det mulig å lage spørsmål og samle svar fra publikum i sanntid.

Dette gjør at vi kan lagre og strukturere all interaksjonen som skjer under en live-presentasjon, og knytte den til både brukere og spesifikke sessions.

6.3.5.2 Relasjoner og struktur

Relasjonene i databasen er bygget opp med foreign keys, som sikrer at data henger riktig sammen:

- En bruker kan eie flere presentasjoner
- En presentasjon kan ha mange slides
- En slide kan inneholde flere elementer

- En poll kan ha flere svar fra ulike brukere

Denne strukturen gjør det enkelt å hente ut relevant data:

- Alle slides i en presentasjon
- Alle elementer på en slide
- Alle svar på en poll i en bestemt session

6.3.5.3 Fleksibilitet med JSON

Vi har også brukt JSONB-felter (f.eks. I `slide_elements` og `slides`) for å lagre mer fleksible data som:

- Design og styling
- Posisjonering av elementer
- Innhold som kan variere i struktur

Dette passer godt med canvas-baserte løsninger ([Fabric.js](#)), hvor elementer ofte har dynamiske egenskaper.

Totalt sett gir databasedesignet oss en god balanse mellom struktur (relasjoner) og fleksibilitet (JSON), noe som passer godt til et interaktivt presentasjonsverktøy med live funksjonalitet.

7 Systemarkitektur

Backend

```
backend/
├─ app/
│   ├─ channels/
│   │   ├─ presentation_channel.rb
│   │   └─ application_cable/
│   │       ├─ channel.rb
│   │       └─ connection.rb
│   ├─ controllers/
│   │   ├─ application_controller.rb
│   │   ├─ spa_controller.rb
│   │   ├─ concerns/
│   │   │   └─ authenticatable.rb
│   │   └─ api/
│   │       └─ v1/
│   │           ├─ auth_controller.rb
│   │           ├─ health_controller.rb
│   │           ├─ polls_controller.rb
│   │           ├─ presentations_controller.rb
│   │           └─ sessions_controller.rb
│   └─ models/
│       ├─ application_record.rb
│       ├─ concerns/
│       │   └─ slide_payload_normalizer.rb
│       ├─ poll.rb
│       ├─ poll_option.rb
│       ├─ poll_response.rb
│       ├─ presentation.rb
│       ├─ presentation_session.rb
│       ├─ session_participant.rb
│       ├─ slide.rb
│       └─ user.rb
```

```
| ├── serializers/
| |   └── presentation_serializer.rb
| └── services/
|   ├── json_web_token.rb
|   └── presentations/
|       └── replace_slides_service.rb
└── config/
    ├── application.rb
    ├── boot.rb
    ├── cable.yml
    ├── database.yml
    ├── deploy.yml
    ├── environment.rb
    ├── puma.rb
    ├── routes.rb
    ├── secret_key_base_resolver.rb
    ├── environments/
    |   ├── development.rb
    |   ├── production.rb
    |   └── test.rb
    └── initializers/
        └── omniauth.rb
└── db/
    ├── schema.rb
    └── migrate/

frontend/
└── src/
    ├── main.tsx
    ├── index.css
    ├── vite-env.d.ts
    ├── app/
    |   └── App.tsx
```

```
├─ features/
│   ├── auth/
│   │   └─ Login.tsx
│   ├── editor/
│   │   ├── PresentationEditor.tsx
│   │   ├── Question.tsx
│   │   ├── SlideThumbnails.tsx
│   │   ├── SlideEmbedOverlays.tsx
│   │   └─ SyncedHostedEmbed.tsx
│   ├── live/
│   │   ├── JoinPage.tsx
│   │   ├── LivePresentation.tsx
│   │   ├── LivePresentationCanvas.tsx
│   │   ├── LivePresentationProjectorShell.tsx
│   │   ├── PresenterSlideViewport.tsx
│   │   ├── SessionLobby.tsx
│   │   ├── usePresenterSlideDeck.ts
│   │   └─ ui/
│   │       ├── LivePresentationAudience.tsx
│   │       ├── LivePresentationPresenter.tsx
│   │       └─ LiveResultsBoard.tsx
│   └─ polls/
│       ├── PollsPage.tsx
│       └─ components/
│           ├── PollCreator.tsx
│           ├── PollResults.tsx
│           └─ PollViewer.tsx
├─ components/
│   └─ ui/
│       ├── badge.tsx
│       ├── button.tsx
│       ├── card.tsx
│       ├── input.tsx
│       └─ label.tsx
```

```
|   ├── mode-toggle.tsx
|   ├── Navbar.tsx
|   ├── textarea.tsx
|   └── theme-provider.tsx
|   ├── hooks/
|   │   ├── usePresentation.ts
|   │   └── useIsMobileDevice.ts
|   ├── lib/
|   │   ├── fabricDefaults.ts
|   │   ├── fullscreenDisplay.ts
|   │   ├── utils.ts
|   │   ├── hostedPlayerControl.ts
|   │   ├── embedUrls.ts
|   │   ├── embedLiveShared.ts
|   │   └── fabricSlideObjects.ts
|   └── services/
|       └── api.ts
```

7.1 Formål

Denne delen beskriver den overordnede systemarkitekturen for løsningen, fra brukergrensesnitt i frontend til backend, databasen og sanntidsfunksjonalitet. Hensikten er å gi et helhetlig bilde av hvordan de ulike komponentene samhandler, samt hvilke teknologiske valg som ligger til grunn for implementasjonen.

7.2 Overordnet arkitektur

Løsningen er implementert som en tredelt web-arkitektur bestående av frontend, backend og database. Frontend er utviklet som en Single Page Application (SPA), og håndterer brukerinteraksjon, redigering av presentasjoner og visning av live innhold. Backend eksponerer et versjonert REST-API som håndterer forretningslogikk, datavalidering og tilgangskontroll, i tillegg til en sanntidskanal for synkronisering under live sessions. Databasen er en relasjonsdatabase som lagrer vedvarende data som brukere, presentasjoner, slides, sessions og svar.

Systemet er designet for å støtte både forberedelse og gjennomføring av presentasjoner. En presentatør kan opprette og redigere innhold i forkant, og deretter starte en live session hvor deltakere kan koble seg til via en unik join kode, QR kode eller link og delta i sanntidsinteraksjoner.

7.3 Systemkontekst

Systemet har to hovedtyper brukere: presentatører og deltakere. Presentatører har utvidede rettigheter til å opprette, redigere og styre presentasjoner, mens deltakere primært konsumerer innhold og bidrar gjennom interaksjoner. Eksterne tjenester inkluderer OAuth-leverandører for autentisering (for eksempel Google eller GitHub), samt nettleserens innebygde støtte for WebSocket kommunikasjon. Systemets kjernefunksjonalitet omfatter presentasjonsredigering, live-avvikling, polls, spørsmål og synkronisering av session tilstand.

7.4 Frontend-arkitektur

Frontend er utviklet med React og bygget ved hjelp av Vite, og består av en komponentbasert struktur som fremmer gjenbruk og modularitet. Applikasjonen er organisert rundt funksjonelle komponenter som representerer ulike deler av systemet, slik som editor, live session og interaksjonsmoduler.

For å oppnå god separasjon av ansvar er API-kall satt i eget servicelag, mens sanntidslogikk er lagt inn i egne hooks. Dette gjør det mulig å isolere kompleks tilkoblings- og eventhåndtering fra presentasjonslaget, noe som bidrar til bedre vedlikeholdbarhet og testbarhet.

Dataflyten i frontend følger et reaktivt mønster: brukerhandlinger utløser kall til enten servicelag eller hooks, som igjen kommuniserer med backend via HTTP eller WebSocket. Respons fra backend oppdaterer applikasjonens tilstand, som igjen fører til re-rendering av relevante komponenter.

Når det gjelder sikkerhet, benytter frontend tokenbasert autentisering. Tilgangstoken lagres i nettleseren og inkluderes i forespørsler til backend. Klientsiden håndterer også grunnleggende validering av input, mens backend har det endelige ansvaret for datavalidering.

7.5 Backend-arkitektur

Backend er implementert i Ruby ved hjelp av Ruby on Rails i en API-orientert konfigurasjon. Systemet følger en lagdelt struktur der controllers håndterer innkommende forespørsler, autoriserer brukere og returnerer JSON-responser. Modeller representerer systemets domeneobjekter og deres relasjoner, mens servicelag benyttes for støttefunksjonalitet som autentisering og token-håndtering.

Sanntidskommunikasjonen håndteres gjennom ActionCable, som muliggjør WebSocket-basert kommunikasjon mellom klient og server. Egen kanalstruktur benyttes for å håndtere hendelser i live-session.

API-et er designet etter REST-prinsipper og versjoneres for å sikre bakover-kompatibilitet.

Ressursorienterte endepunkter benyttes for sentrale entiteter som presentasjoner, slides, sessions og interaksjoner, og et konsistent JSON-format brukes for både data og feilmeldinger.

Autentisering skjer gjennom innlogging eller OAuth, hvor brukeren mottar et token som benyttes i videre kommunikasjon. Autorisasjon er i stor grad basert på eierskap til ressurser, eksempelvis ved redigering av presentasjoner eller styring av live-session.

7.6 Databasearkitektur

Systemet benytter PostgreSQL som relasjonsdatabase. For å sikre robust identifisering av entiteter brukes UUID som primærnøkler i sentrale tabeller. Datamodellen er strukturert rundt kjerneentiteter som brukere, presentasjoner, slides, sessions, og interaksjoner.

Relasjonene mellom entitetene gjenspeiler systemets funksjonalitet: en bruker kan eie flere presentasjoner, en presentasjon består av flere slides, og hver presentasjon kan ha flere tilhørende live-sessions. Poller er knyttet til slides, og svar kan knyttes både til bruker og aktiv session.

Dataintegritet ivaretas gjennom bruk av fremmednøkler, unike indekser og kontrollerte migrasjoner. Dette sikrer konsistens i dataene og legger til rette for videre utvikling av databasen.

7.7 Sanntidsarkitektur

Sanntidsfunksjonaliteten er implementert ved bruk av WebSocket via ActionCable. Klienter abonnerer på en kanal knyttet til en spesifikk presentasjon eller sesjon, og mottar hendelser som reflekterer endringer i systemets tilstand.

```
1 module ApplicationCable
2   class Connection < ActionCable::Connection::Base
3     identified_by :current_user
4
5     # Når en klient kobler til WebSocket, prøver vi å autentisere brukeren basert på en token som sendes i forespørselen.
6     def connect
7       self.current_user = find_verified_user
8     end
9
10    private
11
12    # Hjelpemetode for å finne og verifisere brukeren basert på en token som sendes i forespørselen. You, 2 weeks ago
13    def find_verified_user
14      token = request.params[:token]
15
16      if token.present?
17        decoded = JsonWebToken.decode(token)
18        if decoded && decoded[:user_id]
19          user = User.find_by(id: decoded[:user_id])
20          return user if user
21        end
22      end
23
24      reject_unauthorized_connection
25    end
26  end
27 end
```

Typiske hendelser inkluderer bytte av slide, aktivering av interaksjoner, oppdatering av sessionstatus og publisering av resultater fra polls og spørsmål. Backend fungerer som den autoritative kilden til sannhet, og klientene oppdaterer sin visning basert på mottatte hendelser.

7.8 Nøkkelflyter

Systemet støtter flere sentrale “end to end” flyter. Ved autentisering logger brukeren inn via backend og mottar et token som benyttes i videre forespørsler. Ved opprettelse av presentasjoner redigerer brukeren innhold i frontend, som deretter serialiseres og lagres i databasen via backend.

Når en live-session startes, opprettes en ny session med en unik join kode, og tilkoblede klienter mottar oppdatering i sanntid. Under en live-session kan presentatøren aktivere interaksjoner, hvor deltakernes svar sendes til backend, aggregeres og distribueres tilbake til klientene.

7.9 Deployering og miljø

Systemet er utviklet for både lokal kjøring og produksjonsmiljø. I utvikling kjøres fronten og backend som separate tjenester med en lokal database, mens produksjonsmiljøet benytter tilsvarende arkitektur med miljøspesifikke konfigurasjoner.

Applikasjonen hostes av RubyNor, og deployeres ved hjelp av Kamal. Løsningen er containerisert med Docker, noe som sikrer konsistent kjøring på tvers av miljøer og forenkler utrulling.

I produksjon kjører backend som en API-tjeneste i en container, databasen som en separat tjeneste, og frontend levers som statiske filer eller via en egen server. Kommunikasjon mellom komponentene skjer via HTTP og WebSocket.

7.10 Ikke-funksjonelle egenskaper

Systemet er designet med fokus på ytelse, pålitelighet og sikkerhet. Bruken av WebSocket reduserer behovet for kontinuerlig polling, noe som gir lavere forsinkelse i sanntidsfunksjoner. Databasen er optimalisert med indekser for vanlige oppslag, og frontend benytter effektiv state-håndtering for rask respons i brukergrensesnittet.

Pålitelighet sikres gjennom konsistente databaserelasjoner og robust feilhåndtering i både API og sanntidskanaler. Sikkerhet ivaretas gjennom tokenbasert autentisering og autorisasjon basert på eierskap og kontekst.

8 Sikkerhet

8.1 Autentisering og autorisering

Backend API benytter robuste mekanismer for autentisering og autorisering. Alle sensitive endepunkter er beskyttet ved hjelp av JSON Web Tokens (JWT), som utstedes ved vellykket pålogging eller registrering. Disse tokena signeres med en hemmelig nøkkel og har en utløpsti på 24 timer, noe som sikrer tilgang er tidsbegrenset og kan trekkes tilbake ved å ikke fornye tokena. Systemet støtter også OAuth via Google og Github, slik at brukere kan logge inn sikkert ved hjelp av pålitelige tredjepartsleverandører. Sesjonskapsler er konfigurert med attributtene `httponly`, `secure` og `same_site:lax`, som bidrar til å forhindre cross-site scripting.

8.2 Passordsikkerhet

Brukerpassord lagres aldri i klartekst. I stedet bruker systemet bcrypt-hashingalgoritmen for å sikre passord før de lagres i databasen. Krav til passordkompleksitet håndheves, med minimumslengde på åtte tegn og krav om bruk av store bokstaver, små bokstaver og tall. Når brukeren endrer passordet sitt, krever systemet både det nåværende passordet og bekreftelse av det nye passordet, noe som reduserer risikoen for autoriserte endringer.

8.3 API og datasikkerhet

Backend er konfigurert i API-only modus, noe som reduserer skadeomfanget ved å deaktivere unødvendige middleware og funksjonalitet. Cross-Origin Resource Sharing (CORS) er strengt kontrollert, og kun godkjente kilder får tilgang til API'et. Host-autorisering håndheves også, slik at kun SSL er i bruk og stoler på proxy headere, noe som sikrer at informasjonskapsler og omdirigering alltid er sikre.

8.4 Databasesikkerhet

Databaseskjemaet bruker UUID (Universal Unique Identifier) som primærnøkkel for alle tabeller, noe som gjør det vanskelig å angripe å gjette eller verifisere poster. Alle databaseforbindelser krever SSL, slik at data som overføres er kryptert. Unike indekser brukes for å forhindre dupliserte svar i polls og for å sikre dataintegritet på databasenivå.

8.5 Frontend sikkerhet

På frontend lagres JWT tokens i localStorage og sendes kun i HTTP header, aldri i URL, noe som reduserer risikoen for token-lekkasje. Frontendens API-klient inkluderer retry-logikk som kun forsøker på nytt ved nettverks eller serverfeil, ikke ved klientfeil, noe som bidrar til å forhindre misbruk.

8.6 Sikring i produksjon

Sensitive legitimasjonsopplysninger og hemmeligheter, som database-URL og API-nøkler, lagres i miljøvariabler og aldri i lagret kodebasen. Applikasjonen tilbyr et eget health check-endepunkt for lastbalanser, som ikke eksponerer sensitiv informasjon. Logging er konfigurert til å inkludere request-ID, noe som hjelper med sporbarhet og håndtering av hendelser.

9 Testing

9.1 Formål med testing

Vi brukte PlayWright for E2E (end-to-end-testing) av applikasjonen. Hovedmålet var å verifisere at sentrale brukerflyter fungerer som forventet i en reell nettleser, og å oppdage regresjonsfeil tidlig. E2E tester enhetens- og integrasjonstester ved å validere samspillet mellom frontend, backend og database fra et brukerperspektiv.

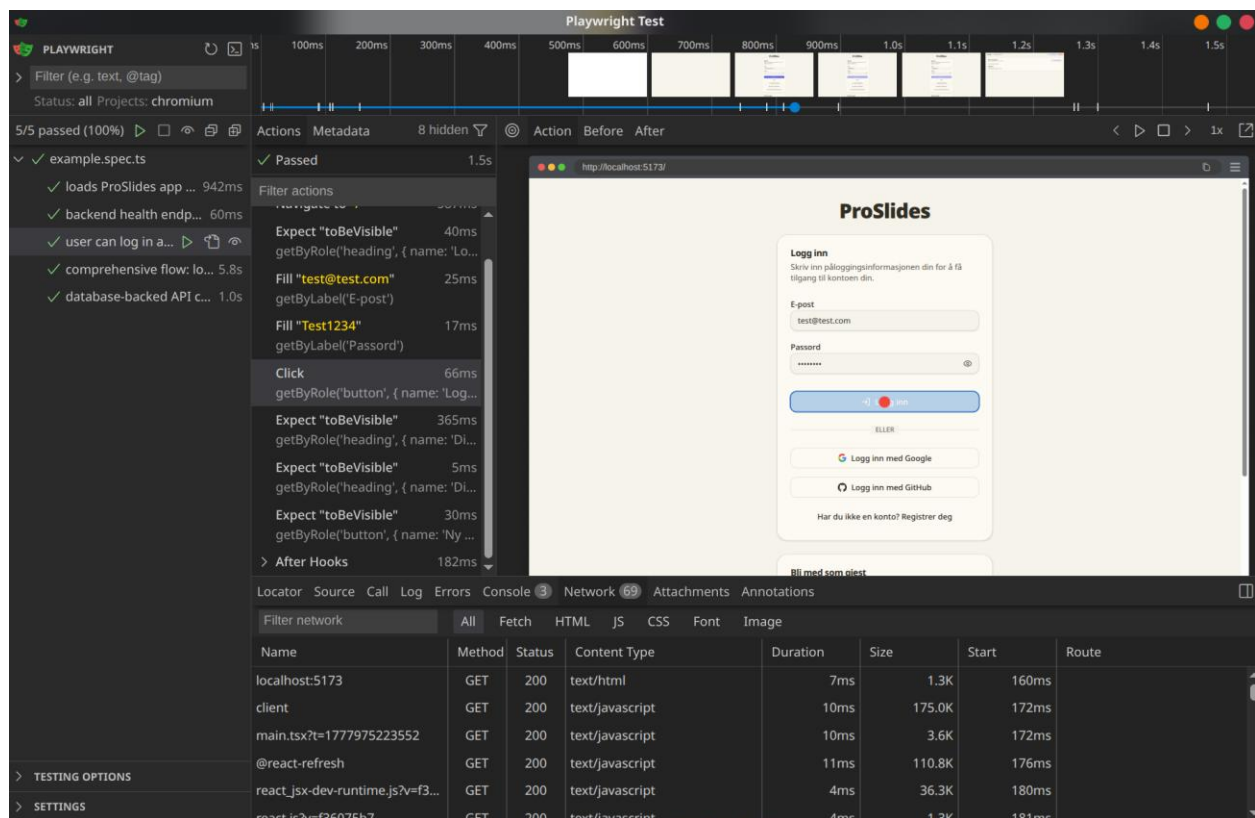
```
Bachelor_Gruppe1 - playwright.config.ts
1 import { defineConfig, devices } from '@playwright/test';
2 const config = {
3   testDir: './tests',
4   testMatch: '**/*.spec.ts',
5   /**
6    * Read environment variables from file.
7    * https://github.com/dotenv/dotenv
8    */
9   // import dotenv from 'dotenv';
10  // import path from 'path';
11  // dotenv.config({ path: path.resolve(__dirname, '.env') });
12
13  /**
14   * See https://playwright.dev/docs/test-configuration.
15   */
16  export default defineConfig({
17    testDir: './tests',
18    timeout: 30 * 1000,
19    expect: {
20      timeout: 20 * 1000,
21    },
22    /** Run tests in files in parallel */
23    fullyParallel: true,
24    /** Fail the build on CI if you accidentally left test.only in the source code. */
25    forbidOnly: !!process.env.CI,
26    /** Retry on CI only */
27    retries: process.env.CI ? 2 : 0,
28    /** Opt out of parallel tests on CI. */
29    worker: process.env.CI ? 1 : undefined,
30    /** Reporter to use. See https://playwright.dev/docs/test-reporters */
31    reporter: 'html',
32    /** Shared settings for all the projects below. See https://playwright.dev/docs/api/class-testoptions. */
33    use: {
34      /** Base URL to use in actions like `await page.goto('')`. */
35      baseURL: 'http://localhost:5173',
36      navigationTimeout: 30 * 1000,
37      actionTimeout: 15 * 1000,
38
39      /** Collect trace when retrying the failed test. See https://playwright.dev/docs/trace-viewer */
40      trace: 'on-first-retry',
41    },
42
43    /** Configure projects for major browsers */
44    projects: [
45      {
46        name: 'chromium',
47        use: { ...devices['Desktop Chrome'] },
48      },
49
50      {
51        name: 'firefox',
52        use: { ...devices['Desktop Firefox'] },
53      },
54
55      {
56        name: 'webkit',
57        use: { ...devices['Desktop Safari'] },
58      },
59
60      /** Test against mobile viewports. */
61      // {
62      //   name: 'Mobile Chrome',
63      //   use: { ...devices['Pixel 5'] },
64      // },
65      // {
66      //   name: 'Mobile Safari',
67      //   use: { ...devices['iPhone 12'] },
68      // },
69
70      /** Test against branded browsers. */
71      // {
72      //   name: 'Microsoft Edge',
73      //   use: { ...devices['Desktop Edge'], channel: 'msedge' },
74      // },
75      // {
76      //   name: 'Google Chrome',
77      //   use: { ...devices['Desktop Chrome'], channel: 'chrome' },
78      // },
79    ],
80  });
```

Figur 1: Oversikt over teststruktur

9.2 Testoppsettet og gjennomføring

Testene ble implementert i Playwright og kjørt både lokalt og i CI-pipeline. Vi benyttet automatiserte testkjøringer for å sikre at endringer i kode ikke introduserer nye feil i eksisterende funksjonalitet.

Testene ble kjørt mot definerte miljøer med faste testdata for å oppnå stabile og reproduerbare resultater.

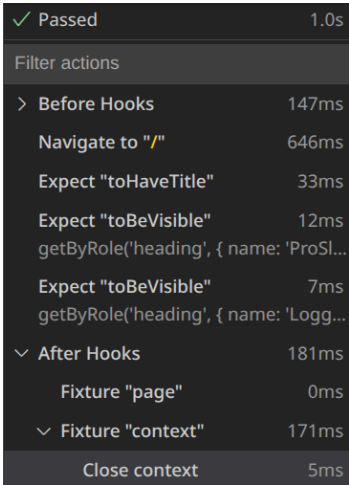


Figur 2: PlayWright konfigurasjon/CI-jobb som har kjørt

9.3 Hva som ble testet

Testene fokuserte på kritiske brukerflows, som for eksempel:

- Innlasting av applikasjon og navigasjon
- Utføring av sentrale handlinger i grensesnitt
- Validering av forventet respons ved gyldige og ugyldige inputs
- Bekreftelse på at data vises korrekt etter brukerhandlinger



✓ Passed	1.0s
Filter actions	
> Before Hooks	147ms
Navigate to "/"	646ms
Expect "toHaveTitle"	33ms
Expect "toBeVisible"	12ms
getByRole('heading', { name: 'ProSl...	7ms
Expect "toBeVisible"	7ms
getByRole('heading', { name: 'Logg...	
✓ After Hooks	181ms
Fixture "page"	0ms
✓ Fixture "context"	171ms
Close context	5ms

Gjennom dette ble både “happy path” og feilscenarioer verifisert.

9.4 Resultater

Resultatene fra testkjøringene viste at testautomatiseringen gir god støtte for kvalitetssikring av leveranser.

Testene identifiserte feil som ellers kunne nådd sluttbruker, spesielt knyttet til brukerflyt og UI-interaksjoner i editor. Etter feilretting ble testene brukt som regresjonssikring for å kontrollere at tidligere fungerende funksjonalitet fortsatt var stabil, og at endringene for feil fungerte.

I tillegg ga PlayWright sine rapport- og feilsøkingverktøy (trace, screenshot, video) raskere feilanalyse når tester feilet. Se Figur 2 for resultat etter fullstendig testing av applikasjon og feilrettinger.

9.5 Vurdering av testmetoden

PlayWright bidrar til økt kvalitet ved at funksjonalitet valideres i et realistisk klientmiljø. Dette reduserer risiko for regresjoner og øker trygghet ved nye leveranser/commits. Samtidig krever E2E-testing vedlikehold av testskript og testdata, og testene har høyere kjøretid enn enhetstester. Likevel vurderes nytteverdien som høy (hvert fall i større programmer/prosjekter), særlig for kritiske brukerflows.

9.6 Testing Milepæl 3

Under fremføringen av milepæl 3 valgte vi å benytte vårt eget presentasjonsverktøy for å få muligheten til å teste løsningen i en større skala enn tidligere, ettersom vi tidligere kun hadde testet med et fåtall personer om gangen. Studentene fikk tilgang til nettsiden og presentasjonen uten større problemer. Nettsiden opplevdes noe treg på grunn av belastning, men alle deltakerne kom seg etter hvert inn i live-session. Bortsett fra belastningsproblemene i starten, da alle skulle inn på nettsiden og koble seg til live-session samtidig, fungerte systemet som forventet. Det var mulig å navigere gjennom presentasjonen, aktivere polls og spørsmål, og få oversikt over svarene fra deltakerne i sanntid.

9.7 Brukerhistorier

Brukerhistoriene nedenfor er utarbeidet basert på en sjekkliste (spørsmål Q1–Q24) som dekker de kritiske delene ved ProSlides. Disse spørsmålene fokuserer på alt fra grunnleggende brukeropplevelse, som registrering, innlogging og feilhåndtering til kjernen i applikasjonen, editoren, opprettelse av interaktive elementer (polls/spørsmål) og den responsive sanntidsfunksjonaliteten for live presentasjoner. Ved å formalisere disse punktene som brukerhistorier sikrer vi at utviklingen har et klart fokus på brukervennlighet og oppfyllelse av de funksjonelle kravene for både presentatører og deltakere.

Se hyperlink i under "ressurser"

Type bruker	Mål (Jeg vil...)	Årsak (Slik at...)	Relaterte spørsmål
Presentatør	Enkelt kunne registrere meg og logge inn med e-post/passord eller via Google/Github, samt enkelt administrere passordkrav og utlogging.	Jeg raskt får tilgang til mine presentasjoner på en sikker måte, samtidig som jeg vet at mine brukerdata er beskyttet.	Q1, Q2, Q3, Q18, Q20, Q21

Ikke-autentisert bruker	Få beskjed om hvilken funksjonalitet som er forbeholdt innloggede brukere.	Jeg forstår begrensningene i applikasjonen når jeg ikke er logget inn.	Q19
--------------------------------	--	--	-----

2. Brukerhistorie: Fri Visuell Utforming (WYSIWYG Editor)

Type bruker	Mål (Jeg vil...)	Årsak (Slik at...)	Relaterte spørsmål
Presentatør	Finne en tydelig måte å starte en ny presentasjon, og enkelt kunne legge til, duplisere og slette slides i sidemenyen.	Jeg kan strukturere innholdet mitt logisk og raskt iterere på presentasjonsflyten.	Q4, Q7
Presentatør	Intuitivt kunne dra-og-slippe og redigere elementer som tekst, bilder og videoer direkte på sliden.	Jeg får full kreativ frihet til å utforme slidenes utseende (WYSIWYG-prinsippet) og lagre presentasjonen uten kompliserte steg.	Q8, Q9

3. Brukerhistorie: Forberede Interaksjon (Polling og Spørsmål)

Type bruker	Mål (Jeg vil...)	Årsak (Slik at...)	Relaterte spørsmål
Presentatør	Enkelt kunne legge til interaktive elementer (polls eller spørsmål) fra verktøy-menyen.	Jeg kan inkludere sanntids publikumsinvolvering som et naturlig element i presentasjonen min.	Q6, Q10
Presentatør	Klart kunne se og forstå forskjellene mellom ulike spørsmålstyper (f.eks. <i>open text</i> vs. <i>single choice</i>).	Jeg velger riktig interaksjonsformat for å samle inn den typen tilbakemelding jeg trenger fra publikum.	Q12
Presentatør	Klart se hvordan man redigerer eller fjerner et interaktivt element fra en slide.	Jeg enkelt kan rette feil eller endre interaksjoner før en live-session starter.	Q11

4. Brukerhistorie: Aktiv Deltakelse (Live Presentasjon og Responsivitet)

Type bruker	Mål (Jeg vil...)	Årsak (Slik at...)	Relaterte spørsmål
Deltaker (Gjest)	Enkelt finne QR-kode eller lenke for å bli med i en livepresentasjon.	Jeg raskt kan koble meg til presentasjonen uten å måtte installere apper eller registrere meg.	Q14
Presentatør	Enkelt starte en live-presentasjon og intuitivt kunne bytte mellom slides.	Jeg har full kontroll over flyten i presentasjonen og sikrer at publikum ser riktig innhold til rett tid.	Q5, Q13

Deltaker (Gjest)	Intuitivt kunne avgi stemme i polls eller svare på spørsmål på min mobiltelefon eller desktop.	Jeg aktivt kan delta i presentasjonen uansett hvilken enhet jeg bruker, slik at min respons blir fanget opp i sanntid.	Q15, Q22, Q23
Brukere (Generelt)	Få tydelig visuell tilbakemelding (feedback) og klare feilmeldinger fra brukergrensesnittet.	Jeg umiddelbart vet at handlingene mine (f.eks. å sende inn et svar) har vært vellykket, eller hva jeg må gjøre for å løse et problem.	Q16, Q17

10 Kritisk vurdering og konklusjon

10.1 Vurdering

Beslutningene vi gjorde i slutten av startfasen har vært med oss hele veien til slutten. Mye av det vi som team avgjorde sammen har blitt videreført i prosjektet. Vi måtte først tilpasse oss det nye kodespråket ruby, rammeverket rails var også noe vi måtte sette oss inn i. Utenom dette tok vi i bruk kjente IDE'er, databaser og andre rammeverk. Det eneste vi har endret på er tidsperiodene i de forskjellige sprintene. Implementasjonen av editoren var det som tok mest tid. Grunnen til dette var at vi måtte bli kjent med nye verktøy som fabric.js og SHADCN. Noe vi kunne gjort annerledes var å sette av litt mer ekstra tid i startfasen til å kunne gjøre oss mer kjent med de nye verktøyene og rammeverket.

Inndelingen av oppgaver kunne også vært løst på en mer oversiktlig måte. Vi regnet med at det kunne bli noe kluss inni mellom med hvem som hadde hva, men dette var forventet når vi valgte å dele inn ansvar og oppgaver på måten vi gjorde det på. Vi kunne satt klarere grenser med hvem som gjør hva, vi kunne gitt en spesifikk person høyest ansvar, men som sagt så ville vi at alle skulle få smake på hvordan arbeidsprosessene og alle aspektene av prosjektet var.

10.2 Kritisk vurdering

En av de første problemene vi støttet på var å tilnærme oss ny kunnskap som Ruby, SHADCN og [fabric.js](#). Når vi nå ser tilbake på det burde vi satt av mer tid til å bli kjent med det nye rammeverket og tilhørende verktøy.

Vi støttet derimot på en del "merge conflicts". Dette er nok grunnet dårlig kommunikasjon og arbeidsfordeling. Vi hadde flere tilfeller hvor mye måtte gjøres på nytt grunnet dette. Filer som ble overwritten og nye filer som ble stashet uheldigvis.

Det første sprintutkastet ble stående gjennom hele prosjektet. Vi la til en ekstra uke på implementasjon (sprint 3). Sprint 3 er den største sprinten grunnet de funksjonelle kravene, og at de funksjonelle kravene er det som definerer nettsiden. Så vi måtte legge en uke til på implementasjon (sprint 3).

Mot slutten av prosjektet ble databasen vår som er hostet av Neon (PostgreSQL) stengt siden vi overskredet grensen av data som er lov å ha på gratis versjonen til Neon. Dette gjorde at vi måtte kjøpe mer plass for å få åpnet databasen, og deretter kunne bruke applikasjonen.

Grunnen til at grensen ble overskredet er for at alt av gjestebrukere, data fra alle live sessions som svar på polls og selve live session ble lagret i databasen for hver gang det ble opprettet en sesjon. Vi implementerte funksjoner i backenden som slettet disse dataene og gjest brukerne etter at en økt blir avsluttet for å redusere data inn i databasen. Dette er noe vi burde sett på og implementert fra starten av for å unngå å måtte oppgradere brukeren vi har inne på vår Neon database.

En ting vi tenkte litt senere på under implementering av spørsmål-funksjonen for live session, er at vi lagrer alle svar på spørsmål i cache i stedet for databasen. Dette er fordi at vi så at det var unødvendig å fylle databasen med svar på spørsmål. Fra starten når vi lagde databasearkitekturen så lagde vi tabeller for polls og svar på polls, dette blir lagret i databasen noe som egentlig kunne blitt lagret i cache. Men dette ble tenkt på veldig sent i prosjektperioden så vi bestemte oss for å la det være.

OBS! Det bør nevnes at Eduroam nettet ikke har vært det beste for kjøringen av webapplikasjonen. Nettet virker noe tregt som gjør at ting ikke får lastet inn eller ting går tregt.

11 Bruk av KI

I dette prosjektet har vi benyttet kunstig intelligens som et støtteverktøy i utviklingsprosessen. KI har blitt brukt til å generere forslag til kode, forklare tekniske konsepter, samt gi inspirasjon til løsninger på konkrete problemer vi har møtt underveis i arbeidet med "ProSlides". Vi har også brukt KI aktivt i kode editoren for å effektivisere kodeskriving, og til hjelp med "bug"-forebygging.

Formålet med bruk av KI er å effektivisere lærings- og utviklingsprosessen, spesielt i situasjoner hvor vi har stått fast eller hatt behov for alternative tilnærminger. KI har fungert som et supplement til egen kunnskap, og ikke som en erstatning for selvstendig arbeid. All kode og alle løsninger som er tatt i bruk i prosjektet er gjennomgått, tilpasset og kvalitetssikret av gruppens medlemmer.

Bruken av KI har dermed bidratt til økt forståelse av fagstoff, samtidig som det har hjulpet oss med å arbeide mer strukturert og effektivt. Vi har vært bevisste på å bruke KI på en ansvarlig måte, og har kun anvendt det som et hjelpemiddel i tråd med gjeldende retningslinjer for utviklingen av dette prosjektet.

11.1 KI Verktøy som har blitt brukt:

- Github Copilot (editor)
 - Dette blir brukt for å effektivisere kodeskriving, hjelp med error-håndtering, og planlegging av funksjoner og implementering.
- ChatGPT (Planlegging)
 - Dette ble brukt i starten av prosjektperioden for å starte å planlegge oppsettet, mappestrukturen og veien videre for prosjektet. Dette effektiviserte starten for prosjektet og gjorde at vi kom oss raskere i gang med utviklingen.
- Cursor AI (editor)
 - Cursor AI ble brukt for effektivisering av kodeskriving, hjelp med funksjoner, planlegging og implementering.

12 Utviklingsverktøy

Gjennom prosjektet har vi tatt i bruk en rekke verktøy for å gjøre prosjektutviklingen lettere og mer behagelig for oss. En rekke verktøy var helt nødvendig å bruke for utviklingen, mens andre var “quality of life” verktøy. Under ser vi en tabell med alle verktøyene som ble tatt i bruk, og en enkel beskrivelse for hvert verktøy.

Navn på verktøy	Beskrivelse av verktøy
VSCode	En lett og fleksibel kodeeditor brukt til utvikling, med støtte for mange programmeringsspråk og utvidelser.
Git	Et versjonskontrollsystem som brukes til å spore endringer i kode og samarbeide effektivt i team.
GitHub	En skybasert plattform for lagring av Git-repositorier, samarbeid, kodegjennomgang og prosjektstyring.
Kamal	Et verktøy for enkel deploy av webapplikasjoner, ofte brukt sammen med Docker for produksjonssetting.
Docker	En plattform for å pakke applikasjoner og avhengigheter i containere for konsistent kjøring på tvers av miljøer.
Neon (PostgreSQL)	En skybasert PostgreSQL-database med fokus på skalerbarhet, ytelse og serverless arkitektur.
GitHub Kanban	Et prosjektstyringsverktøy i GitHub som bruker Kanban-tavler for å organisere oppgaver og fremdrift.
Postman	Et verktøy for å teste og utvikle API-er ved å sende HTTP-forespørsler og analysere responser.
Draw.io	Et nettbasert verktøy for å lage diagrammer som flytskjemaer, systemarkitektur og databasedesign.

Mermaid	Et tekstbasert verktøy for å lage diagrammer (f.eks. Sekvensdiagrammer og flytskjemaer), direkte i kode.
Playwright	Et tekstverktøy for automatisert testing av webapplikasjoner, spesielt for ende-til-ende (E2E) testing.
Discord	En kommunikasjonsplattform brukt til samarbeid i team, med støtte for tekst, tale og videokommunikasjon i organiserte kanaler.
Google meet	En videokonferansetjeneste brukt til nettmøter, presentasjoner og samarbeid i sanntid.
Google docs	En plattform for å kunne redigere og samhandle i sanntid med dokumenter.
Google forms	Et forms verktøy for å kunne lage spørreskjema for testing.
GitHub Copilot	En KI-basert kodeassistent som foreslår kode i sanntid, hjelper med å skrive funksjoner og øker utviklingshastigheten.
Cursor AI	Et AI-basert kodeverktøy bygget på Visual Studio Code som hjelper utviklere med å skrive, redigere og forstå kode raskere.

13 Ressurser

Nedenfor er linker som viser arbeidet gjort i prosjektet. Dette inkluderer kildekode, hovedapplikasjonen og dokumentasjon.

Prosjektweb	https://t3lluz.github.io/proslides-overview/#team
Kildekode	https://github.com/Baitedr/Bachelor_Gruppe1
Hovedapplikasjon	https://slides.rubynor.com/
Spørreskjema (testing)	https://docs.google.com/forms/d/e/1FAIpQLSdpe5ZxRvBg6WImrR8POszsmonqRz96317oOAHzoHzNiqUtjQ/viewform?usp=dialog
Oppgavebeskrivelse	Se vedlegg A
Float Chart	Se vedlegg B
Arbeidskontrakt	Se vedlegg C
Prosjektskisse	Se vedlegg D

13.1 Brukere for testing av applikasjonen

Bruker 1:

- Email: test@test.com
Passord: Test1234

Bruker 2:

- Email: Test1@test.com
Passord: Test123456

13.1.1 Guide/forklaring til testing av hovedapplikasjon

For dem som skal lage bruker, eller allerede har en eksisterende bruker:

1. Gå inn på linken over (hovedapplikasjon)
2. Registrer bruker (Kan velge å bruke google/GitHub, eller lage egen bruker med epost)
3. Logg inn, og du bør se hjemmesiden etter du er logget inn.
4. Trykk på “+ Ny presentasjon” oppe i høyre hjørnet.
5. Du er nå kommet til presentasjonseditoren.
6. Over selve canvasen ser man verktøyet for editoren. (navn på presentasjon, tekststilen, tekst, font, media, former, farger og mer.)
7. Til høyre for canvasen ser man interaksjoner for live session. (Dynamiske variabler, spørsmål for slide og poll for slide).
8. Til venstre er lysbilde seksjonen, der man kan se hvilken slide som har polls og spørsmål ved et lite ikon på toppen. Man kan også endre rekkefølge på lysbilder der ved å dra lysbildet enten ned eller opp.
9. Øverste boks med “logget inn som *navn på brukeren*” ser man lagre knapp (grønn), for lagring av presentasjonen, i tillegg er en “auto save” som lagrer automatisk etter hver endring i editoren.
10. Under canvasen er muligheten for å ha notater for hver slide. Dette er notater for presentatøren under en live-presentasjon.
11. Etter at en presentasjon er lagret kan man velge om man vil starte en live med eller uten lobby. Dette vil si at man kan enten starte for å se hvordan live-presentasjonen vil utforme seg uten å ha publikum til stede (man kan invitere publikum til denne med kode inne i selve presentasjonen om ønskelig), eller starte en live- presentasjon hvor man ankommer en “lobby” først hvor man inviterer publikum med kode eller QR kode for å delta i presentasjonen.
12. For å starte live kan man enten gjøre det fra selve presentasjonseditoren (>Presenter), eller ute på hjemmesiden “start live(med lobby)” eller “start nå(uten lobby)”.
13. I selve live-presentasjonen får presentatøren en egen boks hvor en kan aktivere polls og spørsmål for hver slide.
14. Man navigerer nede til venstre med pilene (fram og tilbake).

15. Det vil komme en resultattavle for både spørsmål og polls sånn at både presentatøren og publikum får en oversikt over alle svar og avstemninger.

For dem som skal være med som gjest:

1. Bruk koden/QR koden som vises i lobby, eller blir tildelt av en presentatør.
2. Hvis man bruker koden så kan man skrive denne inn nederst på login-siden.
3. Etter at gjesten har fylt inn koden eller brukt QR-koden så ankommer gjesten inn i lobby, og deretter venter på at presentatøren skal starte presentasjonen.
4. Det er visning for både mobil og desktop.
5. Gjestene kan svare på spørsmål og polls gjennom presentasjonen, og se resultattavlen.

For dem som skal være med som eksisterende bruker:

1. Når man er logget inn og er på hjemmesiden, så kan man se en blå “bli med live” knapp oppe ved navigasjonsbaren til venstre for “logg ut” knapp.
2. Trykk på den blå knappen og skriv inn koden tildelt av presentatør.
3. Alle som er publikum, har de samme funksjonene (bruker og gjest).
4. Det er visning for både mobil og desktop.

14 Litteraturliste

Aven, T. (u.å.). *Risikoanalyse i Store norske leksikon*. Hentet 5. mai 2026 fra <https://snl.no/risikoanalyse>

Docker Inc. (u.å.). *Docker*. <https://www.docker.com/>

Draw.io. (u.å.). *Diagramming and charting tool*. <https://draw.io/>

Fabric.js. (u.å.). *Fabric.js installasjonsguide*. <https://fabricjs.com/docs/getting-started/installing/>

GitHub. (u.å.). *GitHub Copilot in the IDE*. <https://github.com/features/copilot>

Kamal. (u.å.). *Kamal deploy*. <https://kamal-deploy.org/docs/installation/>

Mermaid.js. (u.å.). *Diagramming and charting tool*. <https://mermaid.js.org/>

Neon. (u.å.). *Neon serverless Postgres*. <https://neon.com/>

OpenAI. (2023). *ChatGPT (23. mars-versjon)* [Stor språkmodell]. <https://chat.openai.com/>

PlayFu. (2022). *PlayFu: Gaming for everyone* [Upublisert bacheloroppgave].

Playwright. (u.å.). *Ende til ende testing av rammeverk*. <https://playwright.dev/docs/intro>

Ruby on Rails. (u.å.). *OAuth*. <https://www.rubyonrails.com/courses/rails-cookie-authentication/lessons/introduction>

Shadcn. (u.å.). *Shad UI installasjonsguide*. <https://ui.shadcn.com/docs/installation>

Squadmates. (2022). *Webapplikasjon for gamere* [Upublisert bacheloroppgave].

Stordalen, J., & Støren, I. (2010). *Bare skriv! Praktisk veiledning i oppgaveskriving* (2. utg.). Cappelen Damm Akademisk.

15 Vedlegg

Vedlegg A - Oppgavebeskrivelse Rubynor

Vedlegg B - Float Chart

Vedlegg C - Arbeidskontrakt

Vedlegg D - Prosjektskisse